

Document Number: P2512R0

Date: 2021-12-23

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [June 9th, 2021](#)
- [June 23rd, 2021](#)
- [July 14th, 2021](#)
- [July 28th, 2021](#)
- [August 25th, 2021](#)
- [September 8th, 2021](#)
- [September 22nd, 2021](#)
- [October 6th, 2021](#)
- [October 20th, 2021](#)
- [November 3rd, 2021](#)
- [November 17th, 2021](#)
- [December 1st, 2021](#)
- [December 15th, 2021](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)

June 9th, 2021

Draft agenda:

- [P2093R6: Formatted output](#)
 - Continue discussion and poll for consensus on answers to the following questions:
 - 1) How should invalidly encoded text be handled when transcoding for the purpose of writing directly to a device interface?
 - 2) Is use of UTF-8 as the literal encoding a sufficient indicator that all input fed to `std::format()` and `std::print()` (including the format string, programmer supplied field arguments, and locale provided text) will be UTF-8 encoded?
 - 3) Is the literal encoding a sufficient indicator in general that all input fed to `std::format()` and `std::print()` (including the format string, programmer supplied field arguments, and locale provided text) will be provided in an encoding compatible with the literal encoding?
 - 4) What are the implications for future support of `std::print("{} {} {} {}"`, `L"wide text"`, `u8"UTF-8 text"`, `u"UTF-16 text"`, `U"UTF-32 text"`)?
- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2093R6: Formatted output](#):
 - No initial discussion was held; the meeting proceeded directly to candidate polls previously [communicated to the mailing list](#).
 - Poll 1 discussion:
 - Zach stated that programmers will expect `std::format()` and `std::print()` to behave the same way.
 - Victor stated that `std::print()` can be implemented using `std::format()`; `std::print()` is intended to be just `std::format()` with additional device dependent transcoding.

- **Poll 1: P2093R6: `<format>` and `<print>` facilities should have consistent behavior with respect to encoding expectations for the format string.**
 - **Attendance: 8**
 - **No objection to unanimous consent.**
- Poll 2 discussion:
 - *[Editor's note: the original poll was "P2093R6: `<format>` and `<print>` facilities should have consistent behavior with respect to encoding expectations for the output of formatters."]*
 - Victor asked for confirmation that the "formatters" term in the poll refers to formatter specializations.
 - Tom confirmed that it does.
 - Zach asked for confirmation that formatters can be user provided.
 - Victor confirmed that they can be.
 - Hubert stated that a desire to bypass encoding constraints will require a concept for binary formatters and a corresponding proposal.
 - Jens expressed a belief that formatters are allowed to be agnostic with respect to use with `std::format()` VS `std::print()`.
 - *[Editor's note: Jens observation prompted the addition of poll 2.2 to confirm matching design intent.]*
 - Victor stated that there is currently no mechanism proposed for a formatter to be informed as to whether it is being used with `std::format()` OR `std::print()`.
 - Zach expressed confusion about the poll.
 - Hubert suggested this poll be deferred until after later polls concerned with the consequences of violating encoding expectations.
- **Poll 2.1: P2093R6: `<format>` and `<print>` facilities should have consistent behavior with respect to encoding expectations for the output of formatters.**
 - Per discussion; poll deferred until after later polls.
- **Poll 2.2: P2093R6: formatters should not be sensitive to whether they are being used with a `<format>` or `<print>` facility.**
 - **Attendance: 8**
 - **No objection to unanimous consent.**
- Poll 3 discussion:
 - *[Editor's note: the original poll was "P2093R6: Regardless of format string encoding assumptions, `<format>` facilities (but not `<print>` facilities) may be used to format binary data."]*
 - Victor stated that support for binary data is a nice capability to have and is needed to match existing uses of `printf()`.
 - Steve noted that this poll is relevant for cases where transcoding is required.
 - Tom agreed and noted that the code author may not be aware of implementation performed transcoding.
 - Jens asked for reasons that a text facility would be used for binary data.
 - Victor responded that `printf()` is often used with binary data and noted that the format string does not necessarily contain text; it might solely contain field

specifiers.

- Tom noted that filenames may be formatted, but might not conform to encoding expectations.
- Steve mentioned having also seen ostream used with binary data.
- Hubert noted again that additional design work would be needed for binary data to be transported through any implicit transcoding performed by `std::print()`.
- Hubert added that control characters can be another source of binary data.
- Zach suggested splitting the poll to address `<format>` and `<print>` separately so as to remove the parenthetical text.
- Zach suggested that there may be a use case for standard formatters for binary data or for a "raw" print interface.
- Victor suggested there may be some misunderstanding; that `std::print()` may be used with binary data with the result that garbage is displayed on the console.
- Hubert politely disagreed due to the lack of an escape mechanism for binary data.
- Jens agreed that some form of a non-text in-band signalling mechanism would be needed.
- Victor clarified that his argument for preserving binary data is for the case where output is directed to a file.
- Hubert noted that poll 3 and poll 10 are related and that consensus for poll 10 will require facilities related to poll 3.
- **Poll 3.1: P2093R6: Regardless of format string encoding assumptions, `<format>` facilities may be used to format binary data.**
 - **Attendance: 8 (1 abstention)**
 - **SF F N A SA**
5 1 1 0 0
 - **Consensus: Strong consensus in favor.**
- **Poll 3.2: P2093R6: Regardless of format string encoding assumptions, `<print>` facilities may be used to format binary data.**
 - **Attendance: 8 (1 abstention)**
 - **SF F N A SA**
2 1 3 1 0
 - **Consensus: Weak consensus in favor.**
 - A: No comment
- Poll 4 discussion:
 - *[Editor's note: the original poll was "P2093R6: `<print>` facilities exhibit undefined behavior when a format string or formatter output does not match encoding expectations."]*
 - Steve expressed a desire for behavior less severe than undefined behavior.
 - Victor expressed discomfort with undefined behavior as well, particularly that the poll applies to all `std::print()` invocations regardless of where the output

is directed.

- Hubert spoke in favor of the poll and noted that this establishes that an implementor or code reviewer can diagnose these cases; that can't happen if behavior is defined.
 - Jens agreed with Hubert, noted the existence of the precondition, and that a violation is "library UB" and therefore less consequential than core language UB.
 - Steve stated in chat: "OK, based on Hubert and Jens's comments, I'll withdraw my objections about UB. I'd like better terminology but this isn't the forum."
 - Jens stated that the paper would benefit from some prose that explains the intended model and that inconsistently encoded data can be stitched together.
 - Jens expressed distaste for preconditions being so specific to a corner case and professed desire for a good programming model.
 - Zach noted similarities with [P1868](#); the worst case outcome is mojibake displayed on the terminal; the damage is limited.
 - Zach stated that either UB or implementation-defined behavior would be fine for now, but that we may desire another failure mode where the behavior is more contained in the future; a behavior mode that reflects that something went wrong, but where the damage is localized.
 - Victor stated that he feels this poll overreaches; that the only concern is with regard to writing to a file vs a terminal and that, in practice, all that should happen is that the data is passed through or that replacement characters are substituted.
 - Hubert noted that files may correspond to special devices; e.g., /dev/tty.
 - Hubert stated that UB is a specification tool and noted that implementors are in a position to distinguish between polls 4 and 5, but that a code reviewer generally cannot.
- **Poll 4: P2093R6: <print> facilities exhibit undefined behavior when an encoding expectation is present and a format string or formatter output does not match those expectations.**
 - **Attendance: 8 (1 abstention)**
 - **SF F N A SA**
2 4 0 0 1
 - **Consensus: Strong consensus in favor.**
 - SA: I think this is too broad and the impact is larger than necessary.
 - **Poll 5: P2093R6: <print> facilities exhibit undefined behavior when an encoding expectation is present and a format string or formatter output does not match those expectations and output is directed to a device that has encoding expectations.**
 - **Attendance: 8 (1 abstention)**
 - **SF F N A SA**
6 0 1 0 0
 - **Consensus: Stronger consensus in favor relative to poll 4.**

- Poll 6 discussion:
 - *[Editor's note: the original poll was "P2093R6: <print> facility implementors are encouraged to provide a run-time means for diagnosing format strings and formatter output that does not match encoding expectations."]*
 - Tom noted that this is not dependent on UB.
 - Hubert agreed.
 - Corentin expressed skepticism that this is implementable.
 - Hubert responded that the binary case is not well supported, but can be done and probably with a reasonable result.
 - Hubert noted that it may be difficult for an implementation of this extension to distinguish the escaped binary data case.
 - Charlie noted that invalidly encoded data can be detected, but that mojibake cannot be.
 - Steve expressed desire for diagnostics for when the data doesn't match the encoding, but not for attempts to match mixed encodings.
 - Zach noted that heuristic warnings can result in false positives and false negatives.
 - Hubert observed that qualitative determination of good vs bad output may require a human.
- **Poll 6: P2093R6: <print> facility implementors are encouraged to provide a run-time means for diagnosing format strings and formatter output that is not well-formed according to the expected encoding.**
 - **Attendance: 8 (1 abstention)**
 - **SF F N A SA**
4 0 2 1 0
 - **Consensus: Consensus in favor.**
 - A: I don't want double validation and this falls outside the standard.
- Tom stated that the next meeting will be in two weeks on June 23rd and that we will complete polling and discuss [LWG 3565](#).

June 23rd, 2021

Draft agenda:

- [P2093R6: Formatted output](#)
 - Finish polling begun at the last telecon.
- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)
 - Discuss and poll the proposed resolution.
- [P2295R4: Support for UTF-8 as a portable source file encoding](#)
 - Review updated wording produced through collaboration between Corentin, Jens, Hubert, and Peter.

- <https://lists.isocpp.org/sg16/2021/04/2353.php>
- <https://lists.isocpp.org/sg16/2021/06/2429.php>

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2093R6: Formatted output](#):
 - PBrett reviewed the polls taken at the last telecon.
 - *[Editor's note: See the [June 9th, 2021](#) summary for the prior polls.]*
 - Tom clarified the intent behind the "encoding expectations" terminology in the polls; it is intended to distinguish cases where there is a dependence on a particular encoding, but without tying that dependence to a particular mechanism for determining the existence of such a dependence. As proposed, the paper currently imposes a UTF-8 encoding expectation when the literal encoding is UTF-8.
 - Hubert expressed being content with poll 5 relative to poll 4 since the determination of what constitutes a device with encoding expectations is left up to the implementation.
 - Hubert noted that it is ambiguous whether a file may constitute a device with encoding expectations and provided `/dev/tty` as an example.
 - Poll 2.1 discussion:
 - Victor stated that `std::format()` does not have an encoding expectation by itself but that string formatters must be encoding aware to honor field width specifiers.
 - Victor added that `std::print()` is special due to transcoding requirements.
 - Hubert noted that these polls address the abstract design extent.
 - Jens stated that, as currently specified, there is no implied encoding expectation, but there may be an expectation for the combined formatter outputs to be consistent.
 - Jens added that the format string might not contribute text to the final result; it might consist solely of field specifiers.

- Jens concluded that concatenation of the output of two formatters that produce differently encoded text might produce text that is not consistently encoded and that nothing is provided to reconcile them.
- Tom agreed and opined that diagnostics would be useful, but that it is not clear how to reconcile that with desired support for binary formatting.
- Victor replied that he doesn't see any problems with combining binary and text and reiterated that the ability to do so addresses real use cases.
- PBrett opined that the `<format>` and `<print>` facilities do not need to be consistent; the only time an encoding expectation should be present is when the output is directed to a device with an encoding expectation.
- Jens asked if that implies that formatters must communicate the encoding of their output.
- Victor replied that use of formatters to combine binary and text data is not dissimilar to existing uses of `std::ostream` or `printf()`; it is up to the programmer to ensure that use of formatters matches the intent.
- Jens asked how a programmer determines what encoding is produced.
- Victor replied that it is determined by the literal encoding.
- PBrett replied that nothing in the standard states that though; not for `std::format()`.
- Charlie stated that the Microsoft implementation assumes Unicode characters for the purposes of field width estimation, but that they could transcode to Unicode if the source encoding was known; but it is not known in general.
- Charlie noted that the arguments passed to formatters are not transcoded.
- Charlie added that format strings frequently consist of only invariant characters; effectively ASCII.
- Charlie cautioned that the encoding of format strings must be known to the implementation in order for format string parsing to not misinterpret trailing code units of multibyte encoded characters.
- Charlie noted that, for log files, it is not necessarily desirable to transcode to the system encoding.
- Corentin portrayed `std::print()` as a two step process of formatting followed by transcoding and stated that there is a precondition on the output device being able to display the text, but noted that such a precondition does not imply a postcondition on `std::format()`.
- Corentin stated that diagnostics would be limited because mojibake is not always detectable.
- Hubert observed that the sentiment for the poll appears to be trending against it, but that we do have desire to avoid surprises with `std::print()`, or at least to say that we want some checking to be implemented.
- Hubert suggested that the model of `std::print()` as a two step process of calling `std::format()` and then printing the result may be too limiting and that a more integrated design that provides `std::print()` more detailed information about formatting outputs may unblock further progress.

- **Poll 2.1: P2093R6: <format> and <print> facilities should have consistent behavior with respect to encoding expectations for the output of formatters.**
 - **Attendance: 9 (1 abstention)**
 - **SF F N A SA**
0 1 1 5 1
 - **Consensus: Strong consensus against.**
- Poll 7 discussion:
 - Victor asked if encouragement would be stated as a note in the standard.
 - Zach responded that LWG prefers normative encouragement of the form, "implementations should do X" and noted that such encouragement does not impose a requirement on implementors.
 - Zach added that it is important to follow Unicode guidelines.
 - Jens asked what the implication is to implementations that cannot implement the encouraged behavior.
 - Zach replied that, as proposed, all implementations would be able to implement it since transcoding is only prescribed for one Unicode form to another.
 - Victor noted that some implementations display a ? rather than a U+FFFD replacement character.
- **Poll 7: P2093R6: <print> facility implementors are encouraged to substitute U+FFFD replacement characters following Unicode guidance when output is directed to a device and transcoding is necessary.**
 - **Attendance: 9 (1 abstention)**
 - **SF F N A SA**
2 5 0 0 1
 - **Consensus: Consensus in favor.**
 - SA: The terminal will already handle this.
 - Tom noted that the device cannot handle this in the case where transcoding is necessary in order to direct the output to the device; e.g., when the device requires UTF-16.
 - Jens noted that specifying that the behavior is undefined but then encouraging a particular behavior is novel.
 - Zach agreed but noted that this is a case of "library UB", so kind of a special case.
- Poll 8 discussion:
 - *[Editor's note: the original poll was, "P2093R6: Neither <format> nor <print> facilities require an explicit program-controlled error handling mechanism for violations of encoding expectations."]*
 - Zach stated that the poll should be framed as a change to the status quo.
- **Poll 8: P2093R6: <print> facilities must provide an explicit program-controlled error handling mechanism for violations of encoding expectations.**
 - **Attendance: 9**
 - **SF F N A SA**
0 0 3 3 3

- **Consensus: Strong consensus against.**
- Poll 9 discussion:
 - [Editor's note: The original poll was "P2093R6: Use of UTF-8 as the literal encoding is sufficient for <format> and <print> facilities to assume that the format string and output of all formatters is UTF-8 encoded."]
 - Tom stated that the poll doesn't make sense as currently worded if formatters are allowed to format binary data.
 - Zach stated that his position may differ for standard formatters vs user provided formatters.
 - Zach added that the proposed heuristic already matches the behavior used to enable field width estimation.
 - Tom disputed the claim that field width estimation depends on the choice of literal encoding.
 - PBrett explained that field width is determined by code point values.
 - [Editor's note: [\[format.string.std\]p11](#) states:

For a string in a Unicode encoding, implementations should estimate the width of a string as the sum of estimated widths of the first code points in its extended grapheme clusters. The extended grapheme clusters of a string are defined by UAX #29. The estimated width of the following code points is 2

...

The estimated width of other code points is 1.
 -]
 - Charlie stated that Microsoft's implementation was designed around the literal encoding at least partially due to current technical limitations in the compiler.
 - Victor stated that the literal encoding is not a perfect indicator, but is the best that we have available.
 - PBrett agreed that we don't currently have anything better.
 - PBrett noted that use of the literal encoding does affect the cases where uses of `printf()` can be simply changed to `std::print()` without potentially unintended behavioral changes.
 - Zach compared use of the literal encoding to use of CMake; the least bad option.
- **Poll 9: P2093R6: Use of UTF-8 as the literal encoding is sufficient for <print> facilities to establish encoding expectations.**
 - **Attendance: 9**
 - **S F F N A S A**
 - **3 1 3 2 0**
 - **Consensus: Very weak consensus.**
 - Corentin commented that LEWG sent these questions back to SG16 for clarification and weak consensus isn't really good enough.
 - PBrett suggested that perhaps use of an encoding tag could garner more consensus.

- Zach reiterated that the status quo is to use the literal encoding to enable width estimation.
- Jens replied that the standard does not connect literal encoding with width estimation.
- [Editor's note: [\[format.string.std\]p10](#) states:

For the purposes of width computation, a string is assumed to be in a locale-independent, implementation-defined encoding. Implementations should use a Unicode encoding on platforms capable of displaying Unicode text in a terminal.

]

- Zach responded that, regardless, implementations are relying on literal encoding.
 - Charlie replied that his implementation should probably be performing width estimation for other encodings like GB18030.
- Poll 10 discussion:
 - [Editor's note: the original poll was "P2093R6: Use of a literal encoding other than UTF-8 is sufficient for `<format>` and `<print>` facilities to assume a particular encoding for the format string and output of formatters."]
 - The weak results for poll 9 obviated the need to conduct this poll.
 - Poll 11 discussion:
 - [Editor's note: the original poll was "P2093R6: Support for implicit encoding conversions will only be possible when an encoding assumption is implicitly or explicitly present."]
 - Victor preempted the poll by volunteering to add prose regarding how future extensions could enable implicit transcoding features.
 - Hubert noted that previous consensus was that `std::format()` and `std::print()` do not require the same encoding expectations.
 - Hubert added that it isn't clear how an implementation might take that into consideration when the implementation intent appears to be to pass the output of a `std::format()` call to a transcoding facility.
 - Corentin stated that LEWG time is more valuable than ours and, since we don't appear to have strong consensus, another meeting seems warranted.
 - Victor agreed with Hubert and Corentin that more common understanding is required.
 - Tom agreed and stated that it seems we are not yet ready to poll forwarding the paper.
 - PBrett pondered how consensus could be improved.
 - Zach suggested that those with positions on the margins could suggest ways in which their positions might be altered.
 - Zach noted that the current proposal and discussion has been on particular technical details and that progress might be made by focusing on, for example, a "Unicode context" as opposed to the choice of literal encoding.

- Hubert requested a clear summary of how the implementation compares to the polls taken.
- Hubert added that he would not oppose moving forward with behavior based on the choice of literal encoding.
- Tom pondered whether Hubert's suggested escape mechanism for binary data would be helpful.
- Victor requested more details on that mechanism, or perhaps a pull request, and stated that he has not seen something that sounds similar implemented elsewhere.
- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)
 - Discussion postponed due to time constraints.
- [P2295R4: Support for UTF-8 as a portable source file encoding](#)
 - Discussion postponed due to time constraints.
- Tom stated that the next meeting will be in 3 weeks, on July 14th.

July 14th, 2021

Draft agenda:

- [P2295R5: Support for UTF-8 as a portable source file encoding](#)
 - Review updated wording produced through collaboration between Corentin, Jens, Hubert, and Peter.
 - <https://lists.isocpp.org/sg16/2021/04/2353.php>
 - <https://lists.isocpp.org/sg16/2021/06/2429.php>
- [P2362R0: Make obfuscating wide character literals ill-formed](#)
- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)
 - Discuss and poll the proposed resolution.

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2295R5: Support for UTF-8 as a portable source file encoding](#)
 - *[Editor's note: D2295R5 was the active paper under discussion at the telecon. The agenda and links used here reference P2295R5 since the links to the draft paper were ephemeral. The published document is expected to differ from the reviewed draft revision as noted below.]*
 - PBrett presented.
 - Peter's presentation slides are available [here](#).
 - The wording was revised based on feedback received from the SG16 mailing list.
 - Any wording changes approved today will appear in the revision of the paper that will be submitted for tomorrow's mailing deadline.
 - Tom noted that the existing wording regarding the introduction of new-line characters for end-of-line indicators only applies to non-UTF-8 encoding schemes with the proposed changes.
 - PBrett and Corentin explained that this is intentional; that end-of-line indicators are relevant for structured text (e.g., data sets), not for source files expressed as a sequence of code units.
 - PBrett and Corentin noted that new-line character sequences will be revisited with [P2348](#).
 - *[Editor's note: A note was added to the final P2295R5 wording to explain that end-of-line indicators are not applicable to UTF-8 encoded source files and that new-line characters separate lines.]*
 - Hubert observed that some of the wording suggestions from the mailing list discussion had not been incorporated.
 - *[Editor's note: Live editing of the proposed wording ensued, the discussion of which is not captured verbatim here. Concerns discussed included use of "encoding scheme" vs "encoding", whether a plural form of "source file" should be used, methods to avoid use of the term "determined", and how to equate the sequence of UTF-8 code units with the elements of the translation character set.]*
 - Mark asked if the proposed wording handles CR/LF new-line sequences.
 - Hubert responded that [P2348](#) will address that concern.
 - **Poll: Forward D2295R5 with wording modifications as discussed to EWG for C++23.**
 - **Attendance: 9**
 - **No objection to unanimous consent.**
- [P2362R0: Make obfuscating wide character literals ill-formed](#)
 - PBrett presented.
 - Peter's presentation slides are available [here](#).
 - Tom noted that the execution wide-character set is not necessarily Unicode; non-encodable characters are possible even when `wchar_t` is 32-bit.

- Charlie noted that Visual C++ is technically not conformant since its 16-bit `wchar_t` is not able to store every possible locale dependent character in a unique `wchar_t` value.
- Hubert explained that ISO C++ does not permit use of a multi-code-unit encoding for wide character and string literals.
- Charlie asked what warning level Visual C++ requires for a warning to be issued for the cases proposed to become ill-formed.
- Corentin responded, W2.
- Tom asked Hubert how his implementation handles the multicharacter case.
- Hubert reported that xLC encodes the last character (like gcc and Clang).
- Wording review ensued.
- Tom requested that the use of "character literal" removed in the proposed wording for [lex.ccon]p2 be restored so that the note states, "... but does not determine the value of non-encodable **character literals** or multicharacter literals. ..."
- PBrett agreed to do so.
- Jens expressed a preference towards revising the paper title to remove the word "obfuscating" in order to avoid projecting bias.
- Tom responded that the title is the author's prerogative, but reported having had a similar reaction to the current title.
- Charlie asked if there is also motivation to make non-encodable character literals and multicharacter literals ill-formed as well.
- PBrett stated that there is and that writing a paper to do so is on his todo list, but that the motivation for ordinary literals is different because they are used and do not suffer some of the problems that the wide variety do.
- **Poll: Forward P2362R0 with title and wording modifications as discussed to EWG for C++23.**
 - **Attendance: 9**
 - **No objection to unanimous consent.**
- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)
 - Deferred to the next telecon due to time constraints.
- Tom announced that the next telecon will be held 2021-07-28 and that the agenda will include [LWG 3565](#) and then [P2348](#).

July 28th, 2021

Draft agenda:

- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)
 - Discuss and poll the proposed resolution.
- [P2348R0: Whitespaces Wording Revamp](#)

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [LWG 3565: Handling of encodings in localized formatting of chrono types is underspecified](#)
 - PBrett presented
 - The standard is underspecified in terms of what happens with localized chrono substitutions
 - Proposed resolution is very narrow; limited to UTF-8 scenarios
 - Hubert: The direction makes sense, but the conversion to UTF-8 may not always be successful given the diversity of possible deployments.
 - Hubert: There should be some form of error handling policy; which one
 - Tom: The assumption is that there may not be characters that are in Unicode?
 - Hubert: No, the implementation may not have a map from the source charset to Unicode.
 - Charlie: Our implementation has `MultiByteToWideChar`, but it behaves in surprising ways for some encodings; some multibyte characters in some encodings may not convert correctly.
 - Charlie: This doesn't permit requesting a non-UTF-8 encoding be used.
 - Victor: If `L` is not specified, then the "C" locale is used and there is no issue.
 - Victor: The proposed wording only applies when `{:L}` is used.
 - PBrett: To clarify, there would be no way to preserve a non-UTF-8 encoding through `std::format()`.
 - Victor: Correct.
 - Charlie: The convention that the literal encoding affect `std::format()` behavior is currently limited; this widens that.
 - Charlie: The other place literal encoding is used is parsing the format string; which makes perfect sense.
 - Charlie: Widening this dependency on the literal encoding is concerning.
 - Charlie: I expect some Windows users to write code with UTF-8 literal encoding but to produce non-UTF-8 output.

- Charlie: This may occur when logging text, the format string may just consist of format specifiers.
- Victor: We also depend on the literal encoding for the "mu" character.
- Victor: Even if text looks like ASCII, it may not be; confusables may be present or line drawing characters.
- Steve: How does the library figure out what the literal encoding is?
- PBrett: Implementation magic; the compiler knows and can communicate it to the library.
- PBrett: Can we just specify that the locale text be transcoded to the literal encoding?
- Charlie: The UTF-8 only solution avoids the need for a large transcoding library. The non-UTF-8 case may not support representation and therefore require/request transliterating.
- PBrett: In an implementation that supports CP1251 as locale, conversion to UTF-8 at least will be needed.
- PBrett: We should allow implementations the flexibility to provide the right result if they know how to.
- Charlie: This is mandating conversion in a specific circumstance; what happens when conversion is lossy? We can't ensure convertibility to all code pages.
- PBrett: The proposed resolution forbids doing the right thing for GB18030, which is able to represent all the characters.
- Charlie: Right, the only encodings that support non-lossy conversion are Unicode ones.
- Charlie: It is reasonable to support EBCDIC here.
- Charlie: With regard to special characters like "mu", you can get mixed encodings regardless.
- Charlie: This differs from width estimation which is always best effort since GUI presentation is not usually known.
- Mark: This does pose a payload requirement on the implementation; not just implementation effort.
- Mark: The overload on locale could be limited to 1; each locale could be required to provide UTF-8 translations.
- Mark: The proposed resolution effectively requires a general purpose transcoding facility.
- Mark: This might be best left to implementation-defined.
- Hubert: There is a desire to allow conversion, but there is also a desire to avoid dependency on the output that locale facilities provide.
- Hubert: The pre-computation method could be intrusive for deployment; limiting localedef to character sets with mapping to Unicode available.
- Hubert: Perhaps guidance is to transcode when encoding information is known.
- Charlie stated in chat: "if you support both `Russian.UTF-8` and `Russian.1251` then this is essentially saying that `format` will treat `Russian.1251` as `Russian.UTF-8` (assuming the actual content of the local facets is the same)"
- PBrett: This is what I was trying to suggest in email.

- PBrett: Only a burden on implementations if they support locale-specific encoding and if the locale specific encoding can be different from the literal encoding.
- PBrett: Implementations that already support many encodings are already burdened with the transcoding facilities.
- Victor: Agree with Peter; the "else" clause in the proposed wording should be relaxed; we should allow, but not require transcoding.
- Steve: For most POSIX system, locales are an open system and may be extended by users (in potentially broken ways).
- Steve: Implementations don't generally own the locale systems, so adding requirements there may not be implementable.
- Steve: But, yes, we should allow implementations to do the best they can; we shouldn't mandate brokenness.
- Charlie: Not a burden if transcoding is only needed for currently supported locales.
- Charlie: Would be a burden if an implementation had to convert between two non-Unicode encodings.
- Charlie: From an overhead perspective, probably not a big deal.
- Charlie: A note may suffice.
- PBrett stated in chat: "'L' = I want to be correct, not fast"
- Corentin: Agree with Peter; avoid specifying transcoding
- Corentin: options are to get output in locale specified, then convert to UTF-8, or to get UTF-8 directly.
- Corentin: Implementations can hack this for `chrono` types; there aren't that many strings involved.
- PBrett: Concerned about implementability since locales may be user-defined; implementations shouldn't have to engage in heroics.
- Hubert: Locale systems have allowances; users can compile their own.
- PBrett: Perhaps limit requirements to locales known by the implementation.
- Hubert: Wording to an implementation-defined set of locales may work here.
- Corentin: There is a limited amount of usefulness that can be extracted here; don't want to put too much effort here.
- Corentin: `std::format()` isn't a great tool for localization; real localization requires swapping the order of fields.
- Jens: Would like to ensure wording is more precise; need to specify which string literal encoding.
- PBrett: Summarizing:
 - 1. Limit the requirement to implementation provided locales.
 - Locales with an implementation-defined set of strings.
 - 2. Permit implementation to "do the right thing"
 - 3. Require "as if" transcoding when the literal encoding is UTF-8.
 - 4. Permit "as if" transcoding when the ordinary literal encoding is not UTF-8.
- Hubert: That seems to reflect consensus, but falls under "as if" rules.
- Tom: Uncertain that we have consensus on dependency of UTF-8 literal encoding.
- Victor: I thought we had consensus on that.

- Mark: Am mildly in favor of requiring this when the literal encoding is UTF-8.
- Hubert: That isn't implementable.
- PBrett: Right, only implementable for locales the implementation provides.
- Charlie: Implementations should be prohibited from transcoding to an encoding that is not Unicode (UCS-2 is not a Unicode encoding in this case).
- Charlie: We don't want transliteration here.
- Charlie: Should require UTF-8, permit UTF-7, UTF-EBCDIC, etc..., prohibit others.
- Hubert: Prior polls had consensus for UTF-8, but not for others. Consensus would likely be similar for other Unicode encodings.
- Tom: Concerned about that consensus.
- PBrett: Concerned about consistency here; trying to rationalize the UTF-8 focus.
- *[Editor's note: Some discussion of poll wording ensued]*
- Corentin: Charlie, why the prohibition to "as if" conversion to other encodings?
- Charlie: The goal is to avoid lossy conversions.
- Corentin: Can we just prohibit lossy conversions?
- Charlie: We could allow cases where the target encoding is not Unicode, but all of the characters are representable.
- Charlie: The concern is wanting to avoid transliteration.
- Corentin: I agree with that.
- **Poll 1: Require implementations to make `std::chrono` substitutions with `std::format` as if transcoded to UTF-8 when the literal encoding *E* associated with the format string is UTF-8, for an implementation-defined set of locales.**
 - **Attendance: 9**
 - **SF F N A SA**
1 6 2 0 0
 - **Consensus: Consensus in favour.**
 - Poll bikeshedding; Tom wants to apply to `wchar_t` cases.
- **Poll 2: Permit such substitutions when the encoding *E* is any Unicode encoding form.**
 - **Attendance: 9**
 - **SF F N A SA**
0 7 2 0 0
 - **Consensus: Consensus in favour.**
- **Poll 3: Prohibit such substitutions otherwise.**
 - **Attendance: 9**
 - **SF F N A SA**
1 3 3 1 1
 - **Consensus: No consensus.**
 - SA: This is an over constraint; should permit implementations to do best effort work.
 - Hubert: This requires invention for the case where a locale is defined outside the implementation without a mapping to the target locale.

- Ran out of time.
- Tom: Next meeting in two weeks, will revisit [LWG 3565](#) if a paper is available; [P2348R0](#) otherwise.

August 25th, 2021

Draft agenda:

- [P2348R0: Whitespaces Wording Revamp](#)
- [P2419R0: Clarify handling of encodings in localized formatting of chrono types](#)
- [LWG 3576: Clarifying fill character in std::format](#)

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Mark Zeren
- Peter Brett
- Steve Downey
- Victor Zverovich

Meeting summary:

- [P2348R0: Whitespaces Wording Revamp](#)
 - Corentin presented
 - Steve: Is "basic source character set" a bug in comment grammar?
 - Corentin: maybe
 - Peter and Steve: Form feeds are used in sources
 - Corentin: no change proposed
 - Hubert: VT and FF don't end comments in clang or gcc. Status quo is they may not be line breaks, although they may be whitespace
 - **Poll 1: Acknowledging that we have limited time available, we support the direction for P2348R0 and encourage further work.**
 - Attendance: 7
 - No objections to unanimous consent
 - Peter: Please bring back the paper rebased on [P2314: Character sets and encodings](#), and add implementation notes.
- [P2419R0: Clarify handling of encodings in localized formatting of chrono types](#)
 - Charlie: Does this permit new things? If so it's appropriate to update feature test macro

- Peter: Would have liked to include recommended practice in the wording
- Charlie: Current wording is 'fine' because it has enough implementation defined wiggle room.
- Hubert: If we are to improve the wording, it might just need to be a note rather than normative
- Victor: Implementation could be in terms of `codecv_t` facet, so it should work
- Charlie: Concern if there's a list of locales, it might be a problem if users customize facets of a locale derived from a system locale.
- **Poll 2: Forward P2419 to LEWG as the recommended resolution of LWG 3565 and with a recommended ship vehicle of C++23.**
 - **Attendance: 7**
 - **S F F N A S A**
4 2 1 0 0
 - **Consensus: Strong consensus in favour.**
- [LWG 3576: Clarifying fill character in `std::format`](#)
 - Charlie: MSVC processes codepoint, preserving the code unit sequence. libcpp stores a code unit. Error handling in MSVC deals with ill-formed sequences transcoding later.
 - Hubert: Clarify as a note grapheme whether a cluster could include ``` or ```
 - Charlie: Implementation difficult, as finding ``` is straightforward, parsing a grapheme cluster is hard.
 - Peter: Doesn't like codepoint as it means combining characters are confusing in source.
 - *[Editor's note: Contribution by Steve not recorded here]*
 - Victor stated in chat: We already talk about grapheme clusters in width estimation
 - Charlie: If we fill with a grapheme cluster, it's the first normative use of EGCs. Some implementation difficulty. Varies over Unicode standard versions in some cases. Users have the ability to customize using formatters. Outside the normal range of use cases. A different format spec/library for multibyte fills? OK with either code unit or codepoint.
 - Corentin: Agree with Charlie, maybe use emoji, but rendering of that is complicated. Doesn't see a use case for combined characters either.
 - Victor: Concerned about implementation experience with grapheme clusters as fill characters. Has had no requests for this functionality. Has had requests for codepoints. Code units would disallow box drawing characters.
 - Peter: We allow EGCs now for width, why shouldn't we allow them as fill characters?
 - Mark: We base on first character of cluster, specified as a heuristic. It's not a layout engine.
 - Charlie: Width is 'should' not 'must' (not mandatory)
 - Victor: We have to restrict the set of fill characters in any case. It might be theoretically better to use grapheme cluster, but has implementation concerns. Way forward is to have a new facility for filling with grapheme clusters.

- Corentin: Question for Charlie and Victor: If we say codepoint now, can we change to grapheme cluster later?
- Charlie: Ict would probably break ABI. Heroic and disgusting hacks would be involved.
- Victor: It would be a break for libfmt.
- Hubert: Are we in agreement that there is an issue with the resolution as presented with it allowing `{}`? Do we need to discuss combining characters?
- Charlie: I don't think so. Not a common use case and not actually totally unreasonable. Could use a `*universal-character-name*`.
- Corentin: No value in protecting user from themselves in something they ask for.
- Peter: Will, "Play stupid games, win stupid prizes," make it into the minutes?
- Victor: Need to prevent characters disallowed by the grammar, but more than that is not necessary.
- Mark: Clarify poll for non-Unicode encoding?
- Charlie: MSVC doesn't treat UCS-2 properly, treats it as UTF-16. Do implementations have to deal with nonsense?
- Peter: This happens after all the other phases of translation
- *[Editor's note: There was some discussion of polling options.]*
- **Poll 3.1: Recommend that the proposed resolution for LWG3576 should be adopted, with the modification that the fill character must not contain '{' or '}' as part of the extended grapheme cluster.**
 - Attendance: 7
 - SF F N A SA
0 1 1 3 2
 - Consensus against.
- **Poll 3.2: The format fill character should be defined as "any codepoint of the literal encoding other than '{' or '}'".**
 - Attendance: 7
 - SF F N A SA
3 3 1 0 0
 - Strong consensus in favour.

September 8th, 2021

Draft agenda:

- [D2348R1: Whitespaces Wording Revamp](#)
- [P2093R8: Formatted output](#)
- [P2361R2: Unevaluated string literals](#)

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Tom: Thank you to Peter and Steve for filling in during my absence.
- PBrett: Consensus from the polls taken during the last telecon held 2021-08-25 and as posted to the mailing list are no longer tentative; no new dissenting opinions were raised.
- [D2348R1: Whitespaces Wording Revamp](#)
 - Corentin: Introduction:
 - Reversed prior intention to classify vertical tab and form feed as new lines.
 - Rebased on top of [P2314R2: Character sets and encodings](#).
 - Would like feedback about support for `\n\r` sequences; support can be provided under implementat-defined behavior.
 - Jonathan Wakely would prefer not to use grammar terms in prose, but unsure how to do that; perhaps Jens can advise.
 - Removed the restriction that non-space characters following a vertical tab and form feed in a single-line comment render the code ill-formed, no diagnostic required; addresses [CWG2002: Whitespace within preprocessing directives](#).
 - PBrett: The goal for now is that the wording reflect the design, it doesn't need to be perfect.
 - Jens: In the new section [lex.whitespaces] there is a horizontal-whitespace that has infinite recursion.
 - Corentin: The intent is to support a sequence of whitespace.
 - Jens: There is a general rule that we use a separate production for sequences of characters.
 - Tom: *h-char-sequence* is such an example.
 - Jens: Yes, and *q-char-sequence*.
 - Jens: The lexical specification for *comment* is problematic due to max munch; nothing prohibits `*/` appearing in the comment. Something is needed to address the intent previously expressed in the removed prose.
 - Jens: In the specification of *d-char*, line-break is not a single character; it may be a sequence and therefore doesn't work following "except".
 - Jens: *basic-s-char* has the same issue.
 - PBrett: Can we use a sequence of *line-break* characters?
 - Jens: No; order matters.

- Jens: [lex.pptoken] hits a conflict between the requirement to capitalize the first word of a sentence and sentences that start with a grammar term; capitalizing the grammar term yields a different term, so the prose must be modified to avoid grammar terms at the beginning of a sentence.
- Jens: Perhaps we should introduce a formal definition of *new-line* to map to the grammar term.
- Jens: There is a general substitution of the *line-break* grammar term for *new-line* in the proposed wording. Can we use *new-line* as the grammar term and not introduce a *line-break* production?
- Corentin: There is a desire to be able to discuss *new-line* abstractly, like in simple escape sequences.
- Jens: I'm wondering if we can avoid that in order to reduce the wording churn.
- Jens: P2314 intentionally did not touch *new-line*; it does update places where a single new-line character is designated; like for simple escape sequence.
- PBrett: Other than for churn; is there motivation to avoid replacing *new-line* with the grammar term?
- Jens: Yes, the changes remove a definition for *new-line* which we assume is needed by library, though I would be happy to be proven wrong.
- Corentin: Library use of *new-line* must refer to the single Unicode new-line character.
- Jens: If *new-line* always designates Unicode new-line, then we can keep *new-line* and use *line-break* for the grammar term.
- Steve: Time format spec supports a %n for new-line character.
- Jens: Could say it is equivalent to \n.
- Jens: There may be interaction with references to the C standard library.
- Corentin: C uses "new-line" as a grammar and library term.
- **Poll 1: Prefer to use the term *new-line* rather than *line-break* in the whitespace grammar production.**
 - **Attendance: 10**
 - **SF F N A SA**
0 0 4 3 1
 - **No consensus for a change.**
- Hubert: With respect to EWG impact; the changes remove a diagnosable issue involving vertical tab and form feed in preprocessor directives.
- Jens: That means we're removing a restriction and that is evolutionary; the changes to [cpp.pre] on page 12 of the paper removes the restriction.
- Corentin: There is no place in the grammar to have a new-line in a preprocessor directive.
- PBrett: Let's have Corentin to resolve this issue and come back with a revised paper.
- [P2093R8: Formatted output](#)
 - Victor presented slides:
 - Slides at <https://github.com/sg16-unicode/sg16-meetings/blob/master/presentations/2021-09-08-p2093r8-presentation.pdf>.
 - LLVM's raw_ostream uses a similar approach.

- Added UB where SG16 requested it if invalid code units are produced by `std::print()`.
- With [P2216: std::format improvements](#), the format string must be known at compile-time and therefore is associated with the literal encoding.
- [LWG3576: Clarifying fill character in std::format](#) was recently resolved through use of the literal encoding.
- If the format string does not match the literal encoding, it could fail to parse.
- Consistency with `std::format` requires locale-independence.
- Consistent with the [{2419: Clarify handling of encodings in localized formatting of chrono types}](#) resolution for [LWG3565](#) where transcoding is performed if the literal encoding is a UTF.
- PBrett: Use of P2419 as a wedge is questionable here since its changes granted permission rather than mandating behavior.
- Victor: We went with more relaxed wording due to concerns over user provided locales; we could strengthen the behavior.
- Hubert: Yes, we had weak consensus for use of literal encoding for UTF-8, but that doesn't imply consensus for more general use.
- Tom: I don't buy the argument that because the format string needs to match literal encoding for compile time processing that that implies the formatted result must be in the same encoding; though production in a different encoding would impose overhead.
- Tom: Use of the literal encoding as required for compile-time parsing of the format string limits this being a precedent for similar use of the literal encoding elsewhere.
- PBrett: We discussed GB18030 recently and wide strings. Victor, are you wedded to this being UTF-8 specific?
- Victor: No. UTF-8 is problematic in practice. Different problems occur for other encodings. Worried about increasing scope though.
- **Poll 2: Use of UTF-8 as the literal encoding is sufficient for <print> facilities to establish encoding expectations.**
 - **Attendance: 9**
 - **SF F N A SA**
2 3 2 1 0
 - **Consensus in favor.**
 - **A: Against rationale: Still concerned that people are not going to use the facility correctly, i.e. end up with mojibake anyway in corner cases that they won't find until later. Would prefer solution that provides a stronger way to associate an encoding with the output, but there isn't an extant proposal to do that.**
- Charlie: I abstained for similar reasons.
- Hubert: We did not read through the minor wording changes in paragraph 31 and it would be good to do so quickly.
- Hubert: Looks pretty good; are we clear that the UB only applies after the first if?
- Hubert: The order of the if statements is not correct; there are subordination issues.

- PBrett: In "If this requires transcoding", it is unclear what "this" refers to.
- Jens: Strike "then" in favor of a comma in "If this requires transcoding then ..."
- Jens: Remove the trademark symbol.
- **Poll 3: Correct the P2093R8 wording for [print.syn].31 to remove ambiguities, and forward P2093 as revised to LEWG with a recommended ship vehicle of C++23.**
 - Attendance: 9
 - SF F N A SA
1 4 2 0 0
 - Consensus in favor.
- [P2361R2: Unevaluated string literals](#)
 - Ran out of time; will discuss next time.
- Next telecon on 9/22 will review D2348R1 subject to a new revision, [P1636 Formatters for library types](#), and [P2361 Unevaluated strings](#).

September 22nd, 2021

Draft agenda:

- [D2348R2: Whitespaces Wording Revamp](#)
- [P1636R2: Formatters for library types](#)
- [P2361R2: Unevaluated strings](#)

Attendees:

- Aaron Ballman
- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Marina Oliveira
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Tomasz Kamiński
- Victor Zverovich

Meeting summary:

- [D2348R2: Whitespaces Wording Revamp](#)
 - *[Editor's note: D2348R2 was the active paper under discussion at the telecon. The agenda and links used here reference P2348R2 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Corentin stated that there are no design change between the R1 and R2 revisions.
 - Tom asked for confirmation that the only known behavioral change is that the VT and FF characters would be well-formed in comments rather than ill-formed no diagnostic required.
 - Hubert responded that the proposal also expands the set of allowed horizontal space characters in preprocessing directives.
 - Aaron asked if there is desire to recommend the proposal as a DR.
 - PBrett responded that there is no need to do so since the changes are effectively specification improvement.
 - Tom asked Hubert if all of the concerns he had raised on the mailing list have been addressed to his satisfaction?
 - Hubert responded that they have been.
 - **Poll 1: Forward D2348R2 to EWG as the recommended resolution of CWG2002 and CWG1655 and with a recommended ship vehicle of C++23.**
 - **Attendance: 12**
 - **SF F N A SA**
2 6 1 0 0
 - **Strong consensus in favor.**
- [P1636R2: Formatters for library types](#)
 - PBrett stated that SG16 is reviewing this paper due to concerns Tomasz raised regarding quoting and localization in the formatting of `std::filesystem::path`.
 - Victor stated that we currently lack the tools to adequately address these concerns now.
 - Victor recommended removing support for `std::filesystem::path` from the paper for now.
 - Victor noted that planned range related enhancements will enable the desired quoting support.
 - PBrett observed that, if explicit support for `std::filesystem::path` is removed, then objects of that type will end up getting formatted as a comma separated list since it models a range.
 - Victor reported plans in place elsewhere to reject use of `std::filesystem::path` as a range.
 - PBrett noted that information can be lost when formatting a path as text.
 - Victor replied that transcoding is possible and that a quoted escape mechanism could be used for portions of a path that would not round trip through a transcoder losslessly.
 - Victor noted that use of the classic locale is a red herring as it has no effect on the output.

- Tomasz noted the existence of two papers that overlap on these design questions.
- Corentin expressed agreement with Victor that support should wait until there is an escaping mechanism available to losslessly preserve path contents in formatted text.
- Charlie noted that there may be cases where replacement characters might be preferred over of of an escaping mechanism that might interfere with further processing of the output.
- Charlie cautioned against including `<format>` in lots of standard library headers since doing so could result in ABI problems if formatter templates are separately compiled.
- Victor opined that `std::format` is effectively a generalized `to_string()` and that every type should be formattable.
- PBindels noted that platform specific knowledge may be required to format paths.
- Charlie remarked that confusion between the literal encoding and the system code page remain possible.
- Charlie noted that Java has the benefit of only needing to compile the code that implements its string type once, but that C++ must do so for every TU that uses it.
- Charlie added that, for Microsoft's implementation, the `<thread>` header includes `<format>` for chrono support.
- Tomasz remarked that it is strange that including `<thread>` results in portions of `<format>` being included, but noted that the standard doesn't require that direct inclusion and that implementations should avoid it.
- Charlie responded that `<thread>` including `<format>` is a quality of implementation issue, but noted that, for formatters, an extern template would be required. However, for `std::format`, the first argument is the format context and it probably can't be declared as an extern template.
- PBindels asked why a platform wouldn't know what encoding is used by the filesystem.
- Charlie responded that file names don't necessarily have an explicitly associated encoding.
- Tom added that a path may have multiple associated encodings if it spans filesystems.
- Charlie further added that additional problems occur with network filesystems that substitute characters for reserved character like ``:`` on Windows.
- PBrett stated that, if the literal encoding is UTF-8, then the associated encoding of `std::string` is nominally UTF-8 and that the `string()` and `u8string()` members of `std::filesystem::path` should return the same content.
- Victor responded that, on Windows, the `string()` member of `std::filesystem::path` returns a string encoded according to the system code page.
- PBrett asked if a similar concern exists for `wchar_t`.
- Steve responded affirmatively; Windows paths are a sequence of 16-bit code units, not UTF-16.
- PBrett suggested a solution like the one adopted for locale dependent chrono fields; if the literal encoding is a UTF, then implementations can convert as best they know

how.

- Victor responded that the same resolution can be used and is simpler because `std::filesystem::path` already offers the necessary encoding conversion functionality.
- PBrett presented a poll option that specified conversion in terms of [\[fs.path.fmt.cvt\]](#).
- Charlie strongly agreed that formatting as if by the `u8string()` member of `std::filesystem::path` is the right thing to do.
- Victor expressed a preference for a solution that preserves all information.
- Tom proposed considering solutions from a text vs binary perspective with a goal to preserve binary representation so as to avoid data loss; programmers can perform conversion to text with their own preferred substitution when desired.
- Victor agreed and noted a desire for a solution that maintains round tripping.
- Tomasz suggested the possibility of multiple formatting options.
- Charlie noted that use of an escape mechanism would solve the problem of conversions between libraries that work in narrow vs wide characters.
- PBrett opined that it sounds like we need an actual proposal for how to format paths.
- PBrett repeated the earlier advice to remove support for `std::filesystem::path` from the paper and encouraged the creation of a new proposal to support it before [P2286](#) is adopted.
- Tomasz stated there is no urgency so long as [P2286](#) precludes handling `std::filesystem::path` as a range.
- **Poll 1: Recommend removing the `filesystem::path` formatter from P1636 "Formatters for library types", and specifically disabling `filesystem::path` formatting in P2286 "Formatting ranges", pending a proposal with specific design for how to format paths properly.**
 - Attendance: 12
 - SF F N A SA
5 5 1 0 0
 - Strong consensus in favor.
- PBrett asked for a volunteer to write the suggested paper.
- Victor volunteered.
- PBrett volunteered to help with wording.
- Mark asked rhetorically if solving the escaping problem also solves the unescaping problem.

- [P2361R2: Unevaluated strings](#)

- Corentin presented:
 - Corentin's presentation slides are available [here](#).
 - Previously, all string literals were converted to the literal encoding in translation phase 5 whether they corresponded to lexical strings or string literal objects.
 - The goal is to prohibit numeric escape sequences and conditional escape sequences in lexical strings, but not in string literals that initialize string literal objects.
 - Support for UCNs and other character escapes is retained for all string literals.

- There is currently implementation divergence regarding when encoding prefixes are or are not allowed.
- Jens noted that the list of unevaluated string literals is missing the literal operator ID case.
- Jens stated that, following [P2314](#), conversion and addition of a null character is now performed during translation phase 7.
- Hubert noted that other proposals are changing nearby wording and that a rebase will likely be needed.
- Hubert observed that wording is missing with regard to how to compare strings in cases for `extern "C"`.
- Corentin replied that he will update the wording.
- Hubert noted that the wording will need to address cases like `extern "\u0043"`.
- Corentin acknowledged that the proposed wording will need some updates.
- Corentin added that SG22 will review the paper soon and that he would like to target C++23.
- Jens identified a grammar ambiguity; *unevaluated-string* and *string-literal* both match *s-char-sequence*.
- Hubert noted that a similar case occurs with *header-name*.
- Jens replied that the *header-name* case can be disambiguated by a preceding `#include` but that the preprocessor cannot disambiguate *unevaluated-string* and *string-literal* in, e.g., `static_assert()`.
- Corentin replied that he'll find a way to address this without modifying the grammar.
- Jens suggested retaining *string-literal* as the lexical term and then handling the different cases where the uses diverge.
- Hubert stated that there are non-diagnostic concerns; for example with `asm` statements.
- Corentin replied that an implementation can do whatever it likes with `asm` strings, such as passing them to an external assembler; the standard doesn't have to address such cases.
- Hubert responded that the proposed change does reduce what the programmer can express, but that an implementation could, for example, do something different with an encoding prefix, issue a warning, and continue.
- Hubert noted that following the introduction of `char8_t`, `u8""` string literals may no longer be accepted in some contexts they previously were.
- Jens remarked that, for string literals, there is a distinct place where encoding conversion is specified; when initializing a string object. For unevaluated string literals, there is no single location.
- Corentin replied that he would work with Aaron to identify a wording solution.
- PBindels asked if the proposal should be recommended as a DR.
- Corentin stated no opinion on the matter.
- Aaron replied that consideration as a DR is questionable.
- PBindels clarified that doing so could make the life of an implementor easier by avoiding any need to fix conformance issues with rejection of encoding prefixes in

earlier standard conformance modes.

- **Poll 3: Acknowledging that we have limited time available, we support the direction for P2361R2 and encourage further work.**
 - **Attendance: 12**
 - **S F F N A S A**
6 5 0 0 0
 - **Strong consensus in favor.**
- Tom announced that the next meeting will be on October 13th.
- *[Editor's note: The next meeting ended up getting moved to October 6th due to scheduling conflicts.]*

October 6th, 2021

Draft agenda:

- [D2460R0: Relax requirements on wchar_t to match existing practices](#)
- [D1885R8: Naming Text Encodings to Demystify Them](#)
 - Discuss and poll issues recently raised on the LEWG and SG16 mailing lists.

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [D2460R0: Relax requirements on wchar_t to match existing practices](#)
 - *[Editor's note: D2460R0 was the active paper under discussion at the telecon. The agenda and links used here reference P2460R0 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Corentin presented:
 - Writing this paper was necessary to make progress on P1885.

- The standard has been out of sync with at least one major implementation for many years.
 - The proposed wording transitions prior core language requirements to library preconditions.
 - PBrett commented that maintaining preconditions in the library wording seems correct, but that the wording should be changed to introduce library UB for characters that are not encodeable in a single code unit.
 - Corentin replied with a desire to agree on the design first and then address wording.
 - Hubert objected to the original paper title ("UTF-16 is standard practice") since UCS-2 is also non-conforming when used as the execution wide-character set if the execution character set contains more characters as happens when UTF-8 is the execution encoding.
 - Hubert agreed with the direction that PBrett suggested.
 - PBrett summarized; the direction is good, some refinement is needed, and some prose is needed to explain why claiming UCS-2 instead of UTF-16 does not suffice to avoid issues.
 - Jens and Hubert clarified that the prose should make it clear that the changes also allow use of UCS-2 when, e.g., UTF-8 is used as the execution encoding.
 - PBrett asserted that the prose should explain how the wording change accomplishes the goals of the paper.
 - PBrett asked if there is an existing core issue for concerns addressed by the paper.
 - Corentin replied that he was unable to find one.
 - Mark verified that there are no active CWG issues that mention UCS-2 or UTF-16.
 - **Poll 1: Add expanded motivation to D2460R0 and forward the paper so revised to EWG with a recommended ship vehicle of C++23.**
 - **Attendance: 10**
 - **S F F N A S A**
5 3 1 0 0
 - **Strong consensus in favor.**
 - Hubert asked if a feature test macro is warranted and noted the existence of `__STDC_MB_MIGHT_NEQ_WC__`.
 - PBrett suggested that SG10 (the feature test study group) review the need for a macro.
 - Tom noted that LEWG should review the paper since it adds library UB where none was possible previously.
 - Tom asked if anyone felt the need to review a revision of this paper in SG16 again.
 - No such desires were raised.
 - Corentin indicated that he will start a mailing list discussion for LEWG.
- [D1885R8: Naming Text Encodings to Demystify Them](#)
 - *[Editor's note: D1885R8 was the active paper under discussion at the telecon. The agenda and links used here reference P1885R8 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*

- Corentin presented:
 - Corentin's presentation slides are available [here](#).
 - The paper goals are limited to tagging known encodings used for interchange, not every possible encoding.
 - There is considerable history, some of it contradictory, mistakes have been made.
 - There are multiple encoding kinds; fixed width vs variable width, single byte vs double byte.
 - Wide interfaces are provided mostly for consistency with char-based interfaces.
 - There are few wide character encodings.
- Hubert disputed the statement that there are few wide character encodings and indicated there are at least as many wide encoding variants as there are ISO-8859 variants.
- Corentin expressed a desire for more information.
- Hubert replied that, for every IBM documented CCSID encoding, there is one two byte and one four byte encoding; the narrow encoding is the odd one that uses a shift-state encoding.
- Hubert noted that documentation is written in terms of character sets that are trivially encoded; encoding schemes are therefore not explicitly documented.
- Tom recommended IBM's "Character Data Representation Architecture" documentation.
- *[Editor's note: Hubert later posted links to related IBM documentation to the SG16 mailing list in an email thread with subject, "Structure of EBCDIC MBCS and wide EBCDIC"; an archive of that message thread is available at <https://lists.isocpp.org/sg16/2021/10/2719.php>.]*
- Hubert noted that he usually consults ICU's converter explorer rather than IBM documentation.
- *[Editor's note: ICU's converter explorer is available at <https://icu4c-demos.unicode.org/icu-bin/convexp>.]*
- Hubert noted that, for `iconv()`, use of the UTF-16 encoding results in BOMs being produced and consumed.
- Jens presented:
 - Jens' presentation slides are available [here](#).
 - An octet is not the same as a byte.
 - The encoding form concept is applicable to non-Unicode encodings.
 - An encoding scheme encodes the output of an encoding form into a series of octets.
 - The "UTF-16" identifier is ambiguous because it may refer to either the encoding form or the encoding scheme.
 - The IANA registry specifies encoding schemes.
- Tom asked if the use case presented for `iconv()` has defined behavior since it involves writing to objects of type `wchar_t` using pointers to `[unsigned] char`.

- PBrett responded that objects of type `wchar_t` can be allocated and then passed to `iconv()` to read or write them.
 - Corentin asserted that the encoding form concept is not useful for users.
 - Tom stated that he remains unclear with regard to behavior for, e.g., UTF-16 in `char` when `CHAR_BIT` is 16.
 - Hubert replied that we take the hand wavy approach and avoid BOMs.
 - Zach stated that, as long as the encoding matches the bits produced, that he is satisfied; there needs to be a 1x1 correspondence between bytes.
 - Jens asserted that UTF-16LE or UTF-16BE should be returned.
 - PBrett replied that programmers won't expect that.
 - Tom suggested that we decide the behavior we want, and then make the wording match that.
 - Jens noted the desire to return UTF-16, but that the definitions in our normative references don't permit that.
 - **Poll 2: The values returned by the `literal()` and ``wide_literal()` functions must indicate the encoding scheme associated with the object representation of ordinary and wide string literals respectively; UTF-16 & UTF-32 are interpreted as having native endianness, and the LE and BE forms are never returned.**
 - Attendance: 10
 - SF F N A SA
4 6 0 0 0
 - Strong consensus in favor.
 - **Poll 3: Notwithstanding the specification in ISO10646, we suggest to return UTF-{16,32} from `literal()` or `wide_literal()` with the understanding that string literals in the compiled program may not actually begin with a BOM and that library facilities [e.g. `iconv()`] may consume a BOM if present.**
 - Attendance: 10
 - SF F N A SA
0 8 1 0 0
 - Strong consensus in favor.
 - **Poll 4: Forward P1885 as revised to incorporate SG-16 feedback on object representation interpretation to LEWG with a recommended ship vehicle of C++23.**
 - Attendance: 8
 - No objection to unanimous consent.
- Tom stated that the next telecon will be October 20th.

October 20th, 2021

Draft agenda:

- D2071R1: Named universal character escapes
 - Add named escape sequences to *universal-character-name* so that these escape sequences can be used everywhere, not just in string literals.
 - Use Unicode rules for matching names rather than requiring exact case-sensitive names.
- [P1885R8: Naming Text Encodings to Demystify Them](#)
 - Continue discussions of issues raised on the LEWG and SG16 mailing lists.
 - Prohibit mapping to IANA encodings when `CHAR_BIT` is not 8?
 - Address special cases for IANA mapping purposes:
 - Is UTF-16 valid for ordinary strings when `CHAR_BIT` is ≥ 16 ?
 - Is UTF-16 valid for wide strings when `CHAR_BIT` is ≥ 16 and `sizeof(wchar_t)` is 1?
 - Is the underlying representation of a wide string required to match an encoding scheme for the encoding form when `sizeof(wchar_t)` is not 1?
 - Limit mapping of wide strings when `sizeof(wchar_t)` is not 1 to other, unknown, and the UCS/UTF variants?

Attendees:

- Charlie Barto
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- D2071R1: Named universal character escapes
 - Steve presented:
 - The most significant change is to make `\N{...}` a *universal-character-name* (UCN); this maintains consistency with the recent addition of `\u{...}` proposed in [P2290 \(Delimited escape sequences\)](#).
 - Implementation experience exists with both Clang and Circle.
 - Both implementations use Corentin's name lookup code.
 - [Editor's note: That code is presumably from Corentin's *ext-unicode-db* repository available at https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp.]
 - Sean Baxter reported a 272K increase in the size of the Circle compiler binary following implementation.

- EWG requested exact name matching.
- Unicode recommends loose matching.
- *[Editor's note: where can one find a citation for this recommendation?]*
- PBrett noted that the paper does not include the rationale for EWG's prior preferences.
- Hubert stated that [P2290 \(Delimited escape sequences\)](#) was discussed in WG14 and noted feedback that curly braces are problematic on EBCDIC systems; though these characters are represented in all EBCDIC code pages customarily used for source code, these characters are not encoded the same way in all such code pages.
- PBrett asked if a replacement syntax would be necessary.
- Hubert replied that an additional syntax would suffice.
- PBrett asked if multiple syntaxes is desirable.
- Jens replied negatively.
- Hubert noted that digraphs can't be used within string literals and that C++ removed support for trigraphs.
- PBrett asked if there is experience in other languages using parenthesis, or perhaps both curly braces and parenthesis.
- PBrett noted that all implementations will be required to support UTF-8 in C++23.
- Hubert acknowledged the UTF-8 requirement and noted that UTF-8 support is useful for source transfer but not so much for native editing since local editors may not support it.
- PBrett observed that an alternate escape syntax could be used in non-UTF-8 encoded source code.
- Hubert acknowledged, but stated that doing so compromises readability.
- Mark asked from chat: "how did format solve this?"
- Hubert responded that `std::format()` is another such problematic case, though even more problematic because of the requirement to process the string according to the literal encoding.
- PBrett noted that, if multiple syntaxes are supported, then people will use what they are most familiar with, probably curly braces due to use in other languages, and end up with non-portable code anyway.
- PBrett opined that supporting a single syntax is the preferred trade off.
- Hubert agreed that, if only one syntax is supported, that it should use curly braces.
- Hubert stated that he was raising the issue because there are programmers that expect code to "just work" even when there are subtle mojibake issues.
- Zach was relieved that there was not a request for a syntax that used only parenthesis.
- Zach commented that we constantly struggle with available syntax, so we should not consume more than is necessary.
- Jens opined that curly braces are members of the basic character set and used in strings elsewhere, so spending extra effort in this case seems unwarranted.
- Jens requested that the paper be updated to note the concern.

- Hubert noted that, in other cases of curly braces in string literals, other escape sequences can be used; that isn't an option for UCNs in string literals though.
- Jens acknowledged and added that UCNs are also restricted to specifying characters that are not members of the basic character set outside of literals.
- Jens suggested that the escape hatch is to not use this feature.
- Tom noted that IBM can continue to use trigraphs.
- Jens agreed with the added observation that such use makes the code non-portable.
- **Poll 1: We should support both `\u{xxxx}` and `\u(xxxx)` (resp. `\N{ABCD}` and `\N(ABCD)`) for better support on EBCDIC systems and others where ``{`` and ``}` are not consistently encoded in the character sets customarily used for source code.**

- **Attendance: 9**

- **S F F N A S A**

1 0 2 4 2

- **Consensus against.**

- Hubert requested that the SG16 chair inform the SG22 chair of this poll result for its relation to [P2290 \(Delimited escape sequences\)](#) and the corresponding proposal for WG14.
- Jens requested that multiple wording options not be present in the paper going forward.
- Steve stated that there are two remaining issues to poll, use of the UAX44-LM2 name matching algorithm and named escape sequences as UCNs.
- Discussion turned to loose name matching.
- Zach commented that code searches become more complicated when loose matching is allowed.
- Jens stated that strong rationale is needed to justify a change of EWG's prior position.
- Tom shared slides presented in Belfast that may have influenced EWG's position on loose matching.
- *[Editor's note: those slides illustrated that matching would succeed with cases like:*

```
"\N{NOBREAKSPACE}"
"\N{NO BREAK SPACE}"
"\N{NO_BREAK_SPACE}"
"\N{NO-B_R-E-A_K-S P A C E}"
```

]

- **Poll 2: Despite previous EWG feedback, we recommend the use of the UAX44-LM2 name matching algorithm.**

- **Attendance: 9**

- **S F F N A S A**

1 3 3 1 1

- **No consensus.**

- **SA: I want to be able to easily grep for names; most other languages don't support loose matching.**

- Steve stated that he will change the paper to remove the recommendation for UAX44-LM2.
- Tom interpreted this poll result as indicating that the compiler size concerns are not motivating.
- Discussion turned to named escape sequences as UCNs.
- Hubert noted that specifying named escapes as a form of UCN raises the issue that formation of a UCN via token pasting results in UB.
- **Poll 3: Named escape sequences should be specified in the language as an alternative form of universal-character-name.**
 - **Attendance: 9**
 - **SF F N A SA**
5 3 0 0 0
 - **Strong consensus in favor.**
- Tom requested that any wording review feedback be sent to the mailing list in advance of the next telecon.
- [P1885R8: Naming Text Encodings to Demystify Them](#)
 - PBrett introduced the topics for discussion:
 - Whether the encoding querying functions should return unknown when CHAR_BIT is not 8.
 - How to handle wide strings for various values of `sizeof(wchar_t)` and CHAR_BIT.
 - Hubert suggested that decisions regarding how to handle CHAR_BIT when it is not 8 may have to be deferred to SG14 for embedded implementations.
 - Zach stated that `sizeof(wchar_t)==1` is problematic when CHAR_BIT is 8.
 - PBrett replied that there is a proposal to lift the restriction that currently requires that `wchar_t` be able to represent all characters of all implementation supported character sets; [P2460 \(Relax requirements on wchar_t to match existing practices\)](#).
 - Jens noted that we have discussed encoding schemes in the context of `wide_literal()` and that BE/LE appropriate results would be expected in that case, but we currently have consensus for a native endian result with no BOM semantics.
 - Jens raised a consistency concern; the paper currently erases the encoding endianness information for the UTF cases, but not for the UCS cases.
 - Jens stated that there are questions about wide-EBCDIC and endianness, but that those encodings don't currently exist in the IANA registry.
 - Jens noted that, at present, the only permissible IANA registered wide encodings when `sizeof(wchar_t)` is not 1 are UTF-16, UTF-32, UCS-2, and UCS-4.
 - PBrett asked Charlie for his impression of what the impact would be of returning UTF-16BE on Windows assuming a bigendian platform.
 - Charlie responded that Windows doesn't support any bigendian platforms, so it wouldn't matter right now; Windows programmers just assume UTF-16LE.
 - PBrett expressed concern about unexpected encoding names being returned and compared using other APIs.

- Hubert observed that programmers may, or may not, want to see UTF-32LE vs UTF-32BE be returned for one Linux system vs another.
- Steve raised the concern of a program externalizing an encoding name as UTF-16 and then providing UTF-16LE text instead of (the expected default of) UTF-16BE.
- Steve mentioned in chat: "UTF-16 generally is supposed to imply BE. In practice it doesn't but, that's an inconsistency."
- Charlie asked in chat: "isn't that just because the network byte order is BE?"
- Jens replied in chat: "Steve: No. ISO 10646 encoding scheme "UTF-16" says "interpret BOM; if none is found, use big-endian"."
- Jens continued in chat: "Steve: iconv does "interpret BOM; if none is found, use host endianness"."
- Tom observed that, in the standard, the wording for string literals is written in terms of code units and encoding form and expressed a belief that programmers tend to work on code units rather than bytes; except for interfaces like `iconv()`.
- Jens replied that previous polls supported an encoding scheme approach in order to support the `iconv()` use case.
- Jens stated that switching to encoding form would be a no-op for ordinary strings.
- Jens added that concern about object representation seems wrong since it is so implementation specific.
- PBrett expressed a desire to work with bytes and that object representation therefore matters for wide strings.
- Hubert acknowledged the present inconsistency and noted the friction with encoding scheme.
- Charlie stated that it is difficult to conceive of cases where the object representation encoding would differ from the native encoding.
- Jens noted that proper byte access would currently require querying native endianness when presented with UTF-16; if the special case for UTF-16 were to be dropped, then behavior would be consistent.
- Tom noted the benefit of being able to use UTF-16BE on little endian systems for encoding tagging purposes.
- Jens observed that friction could be reduced by dropping support for wide strings.
- Tom stated that we should re-poll the special case for UTF-16.
- Tom stated that the next telecon will be November 3rd and that we will plan to poll the special case for UTF-16 for P1885, and possibly look at updated wording for P2071.
- *[Editor's note: since LEWG will be preceding with electronic polling of P1885R9 as is, SG16 will table further discussion of that proposal pending a new paper that argues for changes.]*

November 3rd, 2021

Draft agenda:

- [D2071R1: Named universal character escapes](#)
 - Continue review pending a revision update.
- [P1854R1: Conversion to literal encoding should not lead to loss of meaning](#)
 - New revision review.
- [P2361R3: Unevaluated strings](#)
 - New revision review; we last reviewed this proposal during the [2021-09-22 telecon](#).

Attendees:

- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [D2071R1: Named universal character escapes](#):
 - Tom noted that green highlight is missing in the wording for the feature test macro.
 - Jens stated that the Unicode standard divides name aliases into five types named *correction*, *control*, *alternate*, *figment*, and *abbreviation*, but that ISO 10646 doesn't reflect this partitioning.
 - Steve reported that the aliases described in ISO 10646 appear to be mechanically produced from the Unicode standard's [NamesList.txt](#) file.
 - Steve noted that the names in [NamesList.txt](#) are distinguishable elsewhere in the Unicode DB which is why they are listed in capital letters in ISO 10646; additional names are listed in [NameAliases.txt](#).
 - Jens stated that the ISO 10646 PDF does not retain some information from the Unicode files; for example, ISO 10646 specifies "NO BREAK HERE" as an informative alias for character code point 0083, but omits the "NBH" abbreviation specified in [NameAliases.txt](#).
 - *[Editor's note: The aliases present for character code point 0083 in the Unicode names files are:*

```
NamesList.txt: "NO BREAK HERE"
NameAliases.txt: "NO BREAK HERE" (control)
NameAliases.txt: "NBH" (abbreviation)
```

ISO/IEC 10646:2020 contains (the "=" introducer indicates an informative alias):

```
<control>
= NO BREAK HERE
```

]

- Steve suggested that it may be necessary to refer to the Unicode standard for name aliases.
- PBrett asked what the process would be for requesting changes to the ISO 10646 standard.
- Jens replied that the process is the same for any ISO standard; contact the project editor.
- Hubert suggested that an NB representative could provide contacts for filing an issue with ISO 10646.
- Jens stated that the paper does not make it clear which Unicode name aliases are intended to be usable in these escape sequences.
- Steve stated that ISO 10646 retains some of the Unicode name alias types as normative aliases, but that the rest are informative.
- Steve added that the intended usable names from ISO 10646 are the *associated character name* and each *character name alias* preceded by ✖.
- [*Editor's note: See ISO/IEC 10646:2020 section 34.3, "Character names list". It is currently assumed, but has not been verified, that the normative name aliases (those preceded by ✖) correspond to the aliases present in NameAliases.txt with type **correction**.*]
- Steve asked if there is an expectation to be able to use the name aliases listed in NameAliases.txt with type *control* since the relevant characters do not otherwise have an associated character name.
- [*Editor's note: All control characters are listed in NamesList.txt with "<control>" as the associated character name.*]
- PBrett replied that names for some control characters already appear in the standard and provided "LINE FEED", "SPACE", and "BELL" as examples.
- Jens noted that "SPACE" matches a normative name in ISO 10646, but that the others are problematic; "LINE FEED" is not present though "LINE FEED (LF)" is present as an informative alias, and "BELL" is only present as an informative alias.
- Steve observed that the desired name for control characters might be the first alias listed in ISO 10646.
- Steve asserted that the Unicode *alternate*, *figment*, and *abbreviation* alias types are not stable.
- Tom observed that ISO 10646 appears to combine the Unicode control and abbreviation names to produce informative aliases like "LINE FEED (LF)", "new line (NL)", and "end of line (EOL)" and noted the inconsistent use of case.
- Steve directed the group to the contents of [NamesList.txt](#).
- Jens reported that the names listed in NamesList.txt match those in ISO 10646; it contains the same "LINE FEED (LF)", "new line (NL)", and "end of line (EOL)" aliases noted earlier.
- Steve suggested that NamesList.txt may be parseable; an EBNF specification is present in [Unicode® NamesList File Format](#).
- Tom discovered that section 12 of ISO/IEC 10646:2020 has a list of names for control characters.

- Jens observed that those names are present in a note and are therefore not normative.
- Jens noted that these discoveries indicate that some of the names currently being used in the C++ standard are not correct; "FORM FEED" should be used instead of "FORM FEED (FF)".
- PBrett asked if the note in ISO 10646 can be normatively referenced from the standard.
- Jens replied that a note can be added that matches the note in ISO 10646 for control names.
- Tom noted that ISO/IEC 10646:2020 section 7.4 contains a reference to NamesList.txt; that introduces the possibility of referring to it via ISO 10646.
- Jens stated that the desired names from ISO 10646 are the *associated character name* and *character name aliases* names.
- Tom asked if the issue with missing names is limited to control characters.
- Steve replied that it is.
- PBrett volunteered to take care of getting an issue filed with ISO 10646 so long as someone is available to help define the concern.
- Hubert asked what names other languages support.
- Steve replied that he would research further.
- Tom suggested that we should check what names the existing C++ implementations in Clang and Circle are actually using; those implementations may need refinement.
- Steve replied that both use Corentin's name lookup implementation from https://github.com/cor3ntin/ext-unicode-db/tree/name_to_cp.
- Discussion turned to other specification concerns.
- Jens expressed concern about the standard having a floating reference to ISO 10646 and explained that this is problematic in this case since publication of a new ISO 10646 edition that contains new names would immediately render all implementations non-conforming.
- Steve responded that the standard needs to specify a minimum ISO 10646 edition.
- Jens agreed.
- PBrett asked if the adoption of [P1949 \(C++ Identifier Syntax using Unicode Standard Annex 31\)](#) has this same issue since the introduction of new characters potentially makes new identifiers possible.
- Steve replied that identifiers are at least guaranteed to be stable.
- Tom replied that the character names and name aliases we want are likewise guaranteed to be stable.
- Steve reported that there was originally a desire for a floating reference so that implementations could adopt newer ISO 10646 editions than is specified.
- Jens confirmed that specifying a minimum edition with allowance for implementations to support newer editions is needed and possible thanks to stability guarantees.
- Tom stated that, given that the paper is in line with prior guidance from both SG16 and EWG and that EWG is already scheduled to discuss the new revision on 2021-

11-10, that he believes consensus for the revision exists in SG16 and that no further polls are needed.

- Tom asked for dissenting concerns.
- No such concerns were raised.
- [P1854R1: Conversion to literal encoding should not lead to loss of meaning](#):
 - Hubert suggested that the paper title should be changed to reflect the change the paper actually proposes.
 - Tom asked if Hubert would be willing to submit that feedback to the mailing list and the author.
 - Hubert agreed to do so.
 - *[Editor's note: Hubert did so; the relevant email thread is archived at <https://lists.isocpp.org/sg16/2021/11/2809.php>.]*
- [P2361R3: Unevaluated strings](#):
 - No discussion due to lack of time.
- Tom stated that the next telecon will be 2021-11-17 and will continue discussion of P1854R1 and P2361R3.

November 17th, 2021

Draft agenda:

- [D1854R2: Conversion to literal encoding should not lead to loss of meaning](#)
 - New revision review.
- [P2361R3: Unevaluated strings](#)
 - New revision review; we last reviewed this proposal during the [2021-09-22 telecon](#).

Attendees:

- Aaron Ballman
- Charlie Barto
- Corentin Jabot
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- *[Editor's note: The agenda order was revised to accommodate scheduling conflicts.]*

- [P2361R3: Unevaluated strings](#)

- Corentin introduced the recent wording changes and noted that the *unevaluated-string* production is not matched until after lexing, but is referenced from the wording for the preprocessor line control directive and the `_Pragma` operator as a means to impose constraints on their *string-literal* elements.
- Corentin added that, for `asm` declarations, the only change now is to prohibit an encoding prefix.
- PBrett requested confirmation that this represents a design change.
- Corentin confirmed that it does.
- PBrett asked what the ramification would be if EWG rejected such a change.
- Corentin responded that there is no current implementation experience involving `asm` declarations that use an encoding prefix.
- Corentin added that numeric escape sequences are still allowed in `asm` declarations but that their effect is unknown.
- Aaron noted another change from the prior revision that was inspired by implementation experience; the paper now addresses user-defined literals (UDLs).
- Jens observed that the change to the grammar for the preprocessing line control directive introduces an allowance for use of raw string literals.
- Aaron stated this appears to be an oversight.
- Corentin agreed.
- Jens stated that use of *string-literal* should be avoided for the preprocessing line control directive if the grammar term doesn't apply.
- Aaron noted that this is a pre-existing issue and asked how it should be repaired.
- Jens asked how the C standard handles this.
- Aaron replied that the C standard defines *string-literal* with an optional encoding prefix.
- Corentin stated that the intent was not to enable new syntax, but asked if an allowance for raw strings would be problematic.
- Jens responded that raw strings can contain new lines, but preprocessing directives are line based.
- PBrett noted that such an allowance would introduce a new divergence from C.
- PBrett observed that the current wording discusses *string-literal*.
- Jens agreed that there is an existing issue in that the line control wording discusses *string-literal* where no such production is used.
- Jens suggested retaining the current grammar so as to avoid an unintended change in meaning.
- Corentin agreed to revert the use of *string-literal* in the proposed line control wording and to note the existing issue.
- Jens requested that be included as an editorial note in the wording to ensure CWG considers it during wording review.
- Jens requested that the proposed wording be rebased on the current draft so as to avoid the need for updates to `[lex.phases]` and `[lex.string]`.
- Jens requested that "encoding prefix" be styled as a grammar term in `[dcl.asm]`.

- Jens observed that the user-defined literal operator wording also allows use of raw string literals.
- Jens noted that, in [dcl.link], the comparison of the recognized language linkages includes the quotes thereby requiring that a declaration be written as `extern "\c\"`.
- Corentin reported that Hubert also had a concern that it was not stated how to compare the literal contents in the wording.
- Jens noted that *universal-character-names* (UCNs) can appear in an *unevaluated-string*, but that it isn't clear with respect to the comparison in [dcl.link] when that replacement occurs; `"\u0043"` and `"c"` should be handled equivalently.
- Jens stated that it is unclear why the wording for [cpp.pragma.op] has been updated to strike handling of escape sequences.
- Jens admitted a need to translate UCNs for string literals, but noted that doesn't happen here.
- PBrett observed that doing so could change the meaning of existing code.
- Jens agreed and noted that restoring handling of escape sequences will achieve the desired result; the preprocessing of the destringized string will expand UCNs.
- [P1854R2: Conversion to literal encoding should not lead to loss of meaning](#)
 - *[Editor's note: D1854R2 was the active paper under discussion at the telecon. The agenda and links used here reference P1854R2 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Corentin provided an introduction.
 - PBrett requested that the abstract be updated to summarize the problem the paper addresses, how it is solved, and what the impact is.
 - PBrett suggested that the proposed wording for [lex.ccon] consistently state, "in the literal's associated character encoding".
 - Corentin responded that there is no need to do so since multicharacter literals are no longer subject to use of an encoding prefix; their associated encoding is always the narrow literal encoding.
 - Jens agreed that indirection through an association is not required, but observed that the correct encoding is the "ordinary literal encoding", not the "narrow literal encoding".
 - Jens requested that "encoding prefix" be styled as a grammar term.
 - Discussion ensued regarding the goals of the paper and concluded with the following clarifications:
 - The proposal does **not** intend to prohibit a `c`-char from contributing more than one code unit to the calculation of a multicharacter literal value.
 - The proposal **does** intend to prevent a character literal from being **unintentionally** parsed as a multicharacter literal in visually ambiguous situations.
 - *[Editor's note: Consider 'é' in a UTF-8 encoded source file. If the source file is in Normalization Form C (NFC; `é` is U+00E9 {LATIN SMALL LETTER E WITH*

ACUTE})), then the expression would be an ordinary character literal. However, if the source file is in Normalization Form D (NFD; `é` is U+0065 {LATIN SMALL LETTER E} followed by U+0301 {COMBINING ACUTE ACCENT})), then the expression would be a multicharacter literal. The proposal seeks to avoid such visual ambiguity by restricting the individual written characters in multicharacter literals to those that only contribute a single code unit in the ordinary literal encoding. This suffices to reject the code in the NFD case (U+0301 isn't encodeable as a single code unit in any encodings that are used as the ordinary literal encoding in practice.]

- Corentin agreed to remove the restriction on UCNs from the wording added to the first paragraph of [lex.ccon] since use of a UCN does not produce visual ambiguity.
- *[Editor's note: Thus, the NFD case above can be explicitly written as 'e\u0301'.]*
- Tom announced that the next telecon will be held on 2021-12-01 and that the agenda will include LWG3639 (Handling of fill character width is underspecified in std::format) and further review of P2361 and P1854 pending the availability of new revisions.

December 1st, 2021

Draft agenda:

- [LWG3639: Handling of fill character width is underspecified in std::format](#)
- [P2286R3: Formatting Ranges](#)

Attendees:

- Barry Revzin
- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- *[Editor's note: The agenda order was revised to accommodate attendee schedules.]*
- [P2286R3: Formatting Ranges](#)

- Barry provided an introduction.
 - The goal is to add formatting support for types like tuple, pair, and vector.
 - A sed-like delimiter syntax is proposed to allow for unambiguous formatting of pair and tuple elements.
 - The delimiter syntax may be dropped for now in order to focus on fill and alignment.
 - The delimiter syntax could still be added for a future standard.
- Zach mentioned that the Unicode Bidirectional Algorithm document defines a set of paired brackets that could potentially be used as matched delimiters.
- *[Editor's note: The Unicode Bidirectional Algorithm document is [UAX #9](#). Paired brackets are defined via the UCD Bidi_Paired_Bracket and Bidi_Paired_Bracket_Type properties in [BidiBrackets.txt](#).]*
- Zach provided a brief introduction to how the term "character" gets used. Within the C++ standard, "character" generally means an object of type char, a "code point" represents some part of what we notionally think of as a character, and an "extended grapheme cluster" (EGC) represents a "glyph" or what we visually perceive to be a character.
- Zach stated that we might be able to get away with specifying delimiters as "characters", but noted that such interfaces tend to become regarded as broken later.
- Victor stated that, if the goal is to add some support in C++23, then custom delimiters should be dropped for now given concerns like how use of a digit as a delimiter could lead to problems.
- Corentin agreed with Barry and Victor that custom delimiter support can be postponed in favor of a more comprehensive solution later.
- Charlie argued strongly in favor of use of code points as delimiters given the lack of experience using EGCs in C++20.
- Charlie noted that EGCs do not necessarily correspond to what you might navigate through in a word processor.
- Charlie added that combining code points can be combined with bracket characters.
- Charlie stated that most other languages just use code points for delimiters.
- PBrett expressed concern about the choice of delimiters leading to format strings that are indistinguishable from line noise.
- Barry noted that, without custom delimiters, the only newly required character is ``.``.
- PBrett acknowledged, but noted that a sequence of such characters is needed to navigate range hierarchies.
- Barry agreed, but noted that subrange formatting wouldn't otherwise be possible.
- PBrett suggested that a required custom formatter may be an improvement.
- Barry asked for feedback on two questions.
 - Is everyone happy with use of ``.`` for the debug specifier?
 - Is everyone happy with the described quoting and escaping mechanism for string and character data?
- Victor responded that ``.`` seems ok for the debug specifier.

- PBrett asked if there are other use cases for which ``?` might be desirable.
- Tom noted that ``?` is often used in conjunction with optional data.
- Tom asked why the proposed specifier is called the "debug" specifier.
- Barry responded that "debug" is consistent with Rust's description of its equivalent functionality.
- Barry noted that Python uses "repr" for its equivalent.
- Jens observed that `std::quoted()` already exists for use with iostreams.
- Barry replied that using it would require an additional specifier like `Q`.
- PBindels agreed that the "debug" name for the new specifier is confusing.
- PBrett noted that the "debug" name would not be reflected in written format strings.
- Charlie expressed a preference for "debug" over "repr" so that the latter can be preserved for compiler generated representations.
- Jens asked for a summary of the escaping proposal.
- Barry replied that the intent is to do what `{fmt}` does and deferred to Victor.
- Victor stated that the escaping done by `{fmt}` was recently described in an email to the SG16 mailing list.
- *[Editor's note: that email is archived at <https://lists.isocpp.org/sg16/2021/12/2874.php>.]*
- Victor noted that the paper should be updated to describe what `{fmt}` currently does.
- Jens mentioned that the email states that code points in the range 0 through 0x100 are formatted as hex escape of the form `\xhh`.
- Victor clarified that this substitution only applies to non-printable characters.
- Jens asked what characters are considered non-printable.
- Victor replied that Unicode specifies a non-printable property and that Rust has a non-printable concept.
- *[Editor's note: Unicode does not specify a printable or non-printable property, but does specify many properties from which such properties could be derived.]*
- Tom stated that there appear to be two specification questions:
 - What characters in the code point range 0 through 0x100 are considered non-printable?
 - How are non-printable characters escaped?
- Tom expressed a preference for use of UCN notation for non-printable characters.
- Corentin agreed; use hex escapes for invalid code units and UCN notation for characters.
- Corentin suggested it might make sense to use hex escapes for non-Unicode encodings.
- PBrett asked if it would be a problem to specify UCN notation now, but then switch to [P2290](#) delimited escape sequences later.
- Jens stated that depends on other factors.
- PBrett replied that it therefore seems quite important to make the right decision now.
- Corentin indicated that there is no need to tie the choice of output format to the delimited escape sequences specified in P2290.
- Corentin stated that P2290 will appear in the next EWG electronic voting cycle.

- Victor expressed reluctance towards P2290 delimited escape sequences due to increased verbosity and inconsistency with Rust.
- Victor added that use of brace delimiters with `\x` is unusual.
- PBrett encouraged use of delimited escape sequences for readability benefits.
- Jens asked if it is intended that copy/paste work to produce a string literal that matches the formatted output.
- Barry stated that would be a worthwhile goal.
- Jens noted that it is therefore necessary to avoid potential munging with `\x`; this might require spliced strings.
- Tom noted that such munging is a concern for human consumption as well.
- *[Editor's note: With regard to munging, consider `\xdeface`. Is that a single hex escape, a `\xde` escape followed by `face`, or something in between?]*
- Jens agreed, but noted that a human might expect that only hex escapes with two digits will be produced.
- Jens asserted that the ability to re-parse strongly suggests use of delimited escapes.
- Jens pondered whether the escape mechanism might require an EBCDIC based implementation to transcode to Unicode in order to produce a UCN.
- Jens stated that care is needed that deference to the Unicode DB for a non-printable property not result in a large dependency on the Unicode UCD.
- Jens suggested an implementation should be permitted to escape all non-ASCII characters.
- PBrett suggested that escape sequences could be limited to control characters.
- Corentin reported experience with implementing an `isprintable()` function and noted that it does not require a large table.
- Tom suggested that round tripping of an escaped string output should be possible with use of the `std::scan()` function proposed in [P1729](#).
- Victor posted a link to an `is_printable()` implementation used in `{fmt}` and noted the small size of the tables used.
 - <https://github.com/fmtlib/fmt/blob/master/include/fmt/ranges.h#L268-L395>
- Victor noted that limiting hex escapes to two digits avoids round trip concerns without requiring extra delimiters.
- PBrett requested that the next revision of the paper include discussion of these concerns.
- Corentin asked if the escape mechanism should be exposed as an independent facility.
- Barry suggested that independent facility could just be `std::format()`.
- PBrett observed that a standalone facility could be added later.
- PBrett asked if SG16 should review an updated revision of this paper again.
- Corentin replied affirmatively.
- Jens agreed and noted a need to understand the escape mechanism.
- Jens stated that the paper should also address non-Unicode platforms.
- Corentin noted that, for `wchar_t`, a hex escape with only two digits is insufficient.
- Tom noted that two digits is insufficient for `char` when `CHAR_BIT` is greater than 8.

- Mark observed that the escape facility would be useful for dealing with file names.
- Victor agreed.
- **Poll 0: We recommend using universal character name escape sequences rather than numerical escape sequences for the debug representation of all non-printable characters.**
 - Attendance: 12
 - SF F N A SA
6 2 2 0 0
 - Consensus in favor
- **Poll 1: We recommend using brace-delimited numerical escape sequences as described in P2290 "Delimited Escape Sequences" for 'debug' formatting of invalid codeunits (including lone surrogates).**
 - Attendance: 12
 - SF F N A SA
4 4 1 1 0
 - Consensus in favor
 - A: Delimited hex escape sequences do not exist in C++ yet and are not used elsewhere; but since they will only appear in cases of invalid code units, not SA.
- **Poll 2: We recommend using brace-delimited universal character name escape sequences as described in P2290 "Delimited Escape Sequences" for 'debug' formatting of strings.**
 - Attendance: 12
 - SF F N A SA
3 3 4 0 0
 - Consensus in favor
- [LWG3639: Handling of fill character width is underspecified in std::format](#)
 - Tom provided an introduction.
 - Victor stated that the proposed resolution is somewhat novel and doesn't match what has been implemented in {fmt}.
 - Victor noted the absence of a known use case.
 - Victor added that there is no good solution for when alignment is not possible.
 - Victor noted that option 3 allows changing behavior later.
 - Victor recommended proceeding with option 3; if the estimated width is not 1 then an exception may be thrown or some other UB may occur.
 - Tom asked what current implementations do.
 - Victor responded that {fmt} assumes an estimated width of 1.
 - PBrett argued against option 3 and provided U+3000 {IDEOGRAPHIC SPACE} as an example of a useful fill character with width other than 1.
 - PBrett suggested that an exception could be thrown if alignment requests cannot be met.
 - Zach recommended requiring an estimated width of 1 such that violations are diagnosed as ill-formed at compile-time and result in UB at run-time.

- Zach expressed a desire to avoid paying the cost of checking the estimated width when it will virtually never matter.
- Corentin expressed appreciation for PBrett's use case.
- Corentin stated that the estimated width approach is known not to produce perfect results in general and that he is therefore not very concerned with how this issue is resolved.
- Hubert expressed support for PBrett's use case.
- Hubert noted the current absence of a wording mechanism to determine the number of fill characters to insert.
- Corentin suggested we get implementation experience before proceeding and emphasized that option 3 provides time to do so with the goal of doing better in a future standard.
- PBindels agreed with restriction to an estimated width of 1 now, but with violations resulting in UB so that behavior can be changed later.
- Victor agreed that PBrett's use case is interesting, but asserted that we should not hand wave a solution for it; we should properly explore support for it.
- Tom stated that the next SG16 telecon will be held on 2021-12-15 and will likely revisit LWG3639.
- Tom requested "+1" responses to [Corentin's post](#) to the SG16 mailing list with updates to his [P1854](#) and [P2361](#) papers by anyone that feels these papers are ready to poll forwarding to EWG.
- *[Editor's note: such "+1" responses were provided in response to a [new post](#).]*

December 15th, 2021

Draft agenda:

- [P2361R4: Unevaluated strings](#)
 - Poll forwarding to EWG for C++23.
- [P1854R2: Conversion to literal encoding should not lead to loss of meaning](#)
 - Discuss and poll forwarding to EWG for C++23.
- [D2286R4: Formatting Ranges](#)
 - Review updates since the last telecon.

Attendees:

- Barry Revzin
- Charlie Barto
- Corentin Jabot
- JeanHeyd Meneide
- Jens Maurer

- Peter Brett
- Steve Downey
- Tim Song
- Tom Honermann
- Zach Laine

Meeting summary:

- [P2361R4: Unevaluated strings](#)
 - PBrett explained that SG16 had previously reviewed this paper and that all prior feedback has been addressed.
 - PBrett thanked Corentin for quickly updating the paper in response to the prior review and for soliciting new feedback on the mailing list.
 - PBrett asked if there were any new comments.
 - Tom requested that a table be added to the prose section that summarizes the intended changes; though the effects can be determined from the wording, the impact is subtle with regard to things like where raw string literals are now allowed or disallowed.
 - Corentin agreed to do so.
 - Jens expressed a belief that there are no changes with regard to where raw string literals are and are not allowed.
 - Corentin agreed and noted that there were such changes in a previous revision, but that those changes have been removed.
 - **Poll 0: Forward P2361R4 "Unevaluated strings" to EWG with a recommended ship vehicle of C++23.**
 - Attendance: 9
 - SF F N A SA
2 4 0 0 0
 - Consensus (though with a smaller quorum than is usual due to abstention from late arrivals).
- [P1854R2: Conversion to literal encoding should not lead to loss of meaning](#)
 - Corentin summarized recent changes to improve the motivation and wording and to correct typos.
 - Corentin recalled that this paper was discussed in Belfast and in a recent telecon, but that the paper has not been polled since Belfast.
 - *[Editor's note: Two polls were taken in Belfast as documented in the [minutes for the discussion of P1885](#) The first was a poll to confirm the direction of the paper and the second was to make it dependent on [P1885 \(Naming Text Encodings to Demystify Them\)](#). Both polls had consensus. P1885 was recently approved via electronic polling by LEWG and is expected to be voted on during the next WG21 plenary.]*
 - Corentin explained that the paper proposes two changes:
 - Making non-encodable character literals ill-formed.

- Adding restrictions to the characters that may syntactically appear in multicharacter literals.
- Charlie asked if the proposal will break currently used methods to probe the literal encoding during constant evaluation.
- PBrett replied that we now have a facility that avoids the need for such probing.
- Charlie acknowledged the new facility and that its existence does reduce concerns, but that he still wanted to be sure about what the expectation is.
- Corentin confirmed that such code may be broken and stated that this concern was discussed in Belfast and was the motivation for blocking this paper on adoption of P1885.
- *[Editor's note: Whether such code is broken in practice will depend on what implementors choose to do. The changes require a diagnostic to be produced, but implementors are free to implement that as a warning in which case compilation failure would only occur if warnings are elevated to errors.]*
- Tom noted that P1885 recently passed LEWG electronic polling.
- Corentin asked if the macros added to recent Microsoft Visual C++ releases to reflect the literal encoding are defined regardless of which `/std` options are passed.
- Charlie confirmed that they are.
- *[Editor's note: As of Microsoft Visual C++ version 19.30, the `_MSVC_EXECUTION_CHARACTER_SET` macro is predefined to indicate the code page being used for the literal encoding.]*
- Corentin noted that character probing mechanisms are not particularly reliable.
- PBrett stated that only one implementation is expected to have to change behavior if this proposal is adopted and noted that the implementor in question is aware of the proposal and has so far not objected to the proposed change.
- PBrett reported that prior wording feedback has been addressed.
- Jens read the following proposed addition to `[lex.ccon]`.
 - "If a multicharacter literal contains a *basic-c-char* representing a codepoint that is not encodable as a single code unit in the ordinary literal encoding, the program is ill-formed"
- Jens noted that the difference between *basic-c-char* and *c-char* is that the former excludes escape sequences and asked if the prohibition against escape sequences was intended to apply to *universal-character-names* (UCNs) as well.
- Corentin replied that the design is intended only to apply to visually ambiguous scenarios and that use of a UCN does not create visual ambiguity.
- Jens noted that a UCN is not an escape sequence and that the paper prose discusses escape sequences, but not UCNs.
- Corentin replied that he will update the prose to make it explicit that UCNs are not prohibited.
- Jens pondered whether the previously read wording should state "UCS scalar value" in place of "codepoint".
- Corentin replied that the distinction is not relevant after translation phase 1.

- Jens opined that neither is actually needed and suggested rephrasing as, "... contains a *basic-c-char* that is not encodable as a single code unit ...".
- Corentin agreed to make a change.
- Tom pondered whether the parts of the note removed from [lex.ccon] that continue to be applicable to multicharacter literals should be preserved.
- PBrett pointed out that the note is non-normative and that the relevant parts of it, that multicharacter literals have an implementation-defined value, are normatively specified elsewhere.
- **Poll 1: Modify P1854R2 "Conversion to literal encoding should not lead to loss of meaning" to address wording feedback and forward the paper as revised to EWG with a recommended ship vehicle of C++23.**
 - **Attendance: 10**
 - **SF F N A SA**
3 5 0 0 0
 - **Strong consensus in favor.**
- [D2286R4: Formatting Ranges](#)
 - *[Editor's note: D2286R4 was the active paper under discussion at the telecon. The agenda and links used here reference P2286R4 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Corentin reported that the LEWG chair is skeptical that there is sufficient time available for this proposal to be reviewed and adopted for C++23.
 - Tom reported that both SG9 and SG16 have planned time for review and that, assuming that both SGs forward the paper, further scheduling will be up to the LEWG chair.
 - PBrett reminded the group that SG16 had previously advocated for adding an explicitly deleted format specialization for `std::filesystem::path` to this paper and dropping the support proposed in [P1636R2 \(Formatters for library types\)](#) pending a future paper that addresses `std::filesystem::path` specifically.
 - PBrett stated that he wasn't sure if a later revision of the latter paper actually dropped that support.
 - *[Editor's note: SG16 reviewed P1636R2 during its [2021-09-22 telecon](#); that revision remains the current revision. The poll taken then is recorded in [a comment in the related GitHub tracking issue](#).]*
 - Barry introduced the changes made since the last revision.
 - Hex escapes are now only used for ill-formed code unit sequences.
 - Hex escapes now use delimited escape sequence notation.
 - UCNs are now used for non-printable characters.
 - Jens asked if there is any further intention of reducing scope in order to maintain a target of C++23.
 - Barry replied that the intended scope is what is presented in this revision and that there are no current plans to further reduce scope.

- PBrett asked if consideration was given towards dropping support for the debug format.
- Barry replied affirmatively.
- Jens stated that the escaping behavior needs to address the possibility of lone surrogates.
- Tom asked if the expectation is that lone surrogates would be encoded in UCN notation.
- Jens replied that UCN notation does not permit specifying surrogate code points.
- Jens noted that the escaping behavior is described in terms of code points and that this differs from how string literals are specified; the latter is described in terms of code unit sequences.
- Jens added that specifying escape behavior in terms of code points requires the ability to reconstruct code points from code unit sequences and noted that shift encodings may not have a clearly defined code point space.
- Tom replied that translation to a UCS scalar value would still be possible, but may face implementation challenges.
- Jens noted the dependency on Unicode properties and pondered how that applies to non-Unicode encodings.
- Jens stated that "an implementation-defined equivalent of Unicode properties" could impose a documentation burden.
- PBrett suggested that requirement could be met by documenting a methodology as opposed to an explicit table of equivalent Unicode properties for other character sets.
- Corentin wondered whether newline characters should always be escaped.
- Corentin noted that there are design questions regarding whether unassigned code points and private use area (PUA) characters should be escaped.
- Corentin suggested that PUA characters should probably be escaped but that it is less clear how unassigned code points should be handled.
- Corentin wondered what the performance cost would be for the requirement to check the `Grapheme_Extend` property for characters at the start of a string.
- Corentin suggested that it may be desirable to specify escape behavior in terms of conversion to Unicode to ensure consistent behavior across implementations.
- Tom asked how it was determined that the `z` (Separator) and `c` (Other) values of the `General_Category` property suffice to define printable characters.
- Corentin replied that those properties exclude all control, separator, and unassigned characters.
- Corentin noted that there is a design decision to be made regarding which separators should be considered printable.
- Corentin added that there is a trade off between getting a "right" result and potentially requiring a possibly large table of character properties.
- Tom asked if the lookup for the `Grapheme_Extend` property is intended to identify combining characters for which a base character is not available to combine with.
- Corentin confirmed that is the intent.

- Charlie asserted a need for further elaboration of what is meant by "a code unit that is not a part of a valid code point".
- Zach asserted that PUA characters should not be escaped and that they should be usable in the same manner as any other printable character.
- Zach stated that Unicode specifies how sequences of invalid code units should be handled and that processing them should be left to QoI.
- *[Editor's note: See the "Constraints on Conversion Processes" and "U+FFFD Substitution of Maximal Subparts" sections of 3.9, "Unicode Encoding Forms", in [chapter 3 of Unicode 14.0](#) for Unicode recommendations regarding handling of ill-formed code unit sequences.]*
- Tom stated that his understanding is that the intent is to preserve the values of all bytes that contribute to an invalid code unit sequence.
- Charlie mentioned that the Unicode standard refers to the [WhatWG encoding standard](#) for handling of ill-formed code unit sequences.
- *[Editor's note: It does so in the "U+FFFD Substitution of Maximal Subparts" section mentioned in the previous note.]*
- Charlie noted a design question; how are invalid code unit sequences delimited?
- Charlie suggested that it might be ok to discontinue consuming text after an invalid code unit sequence.
- Charlie asserted a requirement for wording to prohibit considering code units following an invalid code unit sequence as themselves being part of the invalid code unit sequence if they could signify the start of a potentially valid code unit sequence.
- *[Editor's note: This is consistent with guidance in the "Constraints on Conversion Processes" section mentioned in a previous note.]*
- Corentin asserted that replacement characters are not particularly helpful when trying to diagnose unexpected output; the actual byte or code unit values are needed.
- Corentin stated that further discussion regarding handling of ill-formed code unit sequences is needed.
- PBrett indicated that consensus for how to handle invalid code unit sequences is not yet clear and that there exists a design question of whether to emit replacement characters or preserve code unit values via hex escapes.
- PBrett suggested it may be worth stating in [SD-8](#) that debug formatting is not stable.
- Corentin noted that, because Unicode character properties are not stable, that we can't commit to stability anyway.
- PBrett requested that Barry submit the draft revision as a P paper.
- Barry agreed to do so, but reported that he had already edited it in response to the discussion.
- Corentin asked if the group has concerns regarding handling of non-Unicode encodings.
- PBrett replied that he would like to see wording, but that we are short on time.
- **Poll 2: Modify D2286R4 to address design feedback, and forward the published paper as revised to LEWG with a recommended ship vehicle of C++23.**

- Attendance: 10
- SF F N A SA
3 3 2 0 1
- Consensus.
- N: Lack of wording.
- SA: Lack of wording; concerned that there will be subtle issues that won't become apparent until wording is available.

- Tom announced that the next telecon will be held 2022-01-12 and that the agenda is expected to include review of an updated revision of [P2286 \(Formatting Ranges\)](#), review of an updated proposed resolution for [LWG3639 \(Handling of fill character width is underspecified in std::format\)](#), and [LWG3576 \(Clarifying fill character in std::format\)](#), and/or initial review of [P2491R0 \(Text encodings follow-up\)](#) and [P2498R0 \(Forward compatibility of text_encoding with additional encoding registries\)](#).